

Nested YOSO MPC: Near Optimal Resilience Without an MPC Setup

Ittai Abraham¹, Eli Chouatt², Ivan Damgård³, Yossi Gilad², Gilad Stern⁴, and
Sophia Yakoubov³

¹ Intel Labs

² Hebrew University

³ Aarhus University

⁴ Tel Aviv University

Abstract. You Only Speak Once (YOSO) secure Multi-Party Computation (MPC) provides scalability and adaptive security by distributing the computation across a sequence of anonymous committees. This relies on *role assignment*, which sets up the infrastructure necessary for the members of one committee to send secret messages to members of subsequent committees without knowing their identities. Existing approaches to role assignment suffer from significant limitations: high broadcast complexity, reliance on secure computation (which creates a circular dependency), or the requirement of an overwhelming honest majority among available nodes in order to guarantee output delivery in the MPC.

In this work, we introduce Nested YOSO MPC, which overcomes all of these drawbacks by departing from the traditional threshold adversary structure. We design our MPC to run over a sequence of size- m^2 committees, each composed of m subcommittees of size m . This MPC is secure (with guaranteed output delivery) as long as most subcommittees on each committee have an honest majority. Our efficient, setup-free role assignment protocol guarantees an honest majority on most subcommittees as long as $(\frac{1}{2} + \epsilon)N$ of the available participants are honest, where ϵ is a constant and N is the total number of available participants.

As a complementary contribution, we prove that *no* MPC with polylogarithmic (in N) broadcast complexity can rely on $\frac{N}{2} + o(\frac{N}{\text{polylog}(N)})$ honest parties, indicating that our construction is near-optimal in terms of the number of corruptions it can tolerate.

Table of Contents

Nested YOSO MPC: Near Optimal Resilience Without an MPC Setup . . .	1
<i>Ittai Abraham, Eli Chouatt, Ivan Damgård, Yossi Gilad, Gilad Stern, and Sophia Yakoubov</i>	
1 Introduction	2
1.1 Technical Overview: Upper Bounds	5
1.2 Technical Overview: Lower Bounds	8
1.3 Paper Outline	8
2 General Role Assignment Functionality	9
3 Role Assignment Protocol	12
3.1 Protocol	12
4 Compiling Abstract to Natural	15
5 Building Blocks for MPC	16
5.1 Zero-Knowledge	16
5.2 Linear Integer Secret Sharing	17
5.3 Linearly Homomorphic Encryption	20
6 MPC Subprotocols	22
6.1 Decrypt	22
6.2 Reshare	23
6.3 Secure Multiplication	24
7 CDN MPC	25
7.1 Functionalities	25
7.2 Protocols	26
A Definitions for Secure Multiparty Computation	31
B Building Blocks for Role Assignment	32
B.1 Random Beacon	32
B.2 Verifiable Random Function (VRF)	35
B.3 Key-Message Non-Committing Encryption (KMNCE)	36
C Proof of Theorem 1	37
D Lower Bounds	40
D.1 Proof	42

1 Introduction

You Only Speak Once (YOSO) secure Multi-Party Computation (MPC) [Gen+21a] is a scalable approach to MPC in the presence of an adaptive adversary (which, unlike a static adversary, can corrupt new parties throughout the computation). In a YOSO MPC protocol with N available participants, the computation is carried out by a sequence of *committees*. Scalability comes from the fact that the total number of committee members (across all committees) can be much smaller than N , allowing the total number of bits broadcast to be sublinear in N . In each round, the committee associated with that round speaks. The members of

each committee remain anonymous until they speak, and become irrelevant once they have spoken, since they have fulfilled their committee role. This makes it impossible for an adaptive adversary to target key participants for corruption.⁵

In order to transmit secret state from the i th committee to the $(i + 1)$ st (while allowing the members of the $(i + 1)$ st committee to remain anonymous even to members of the i th committee), YOSO MPC protocols rely on *receiver-anonymous communication channels* (RACCs). An RACC can be realized as an encryption-decryption key pair where the encryption key is public and the secret decryption key is held by an anonymous participant. YOSO MPC protocols can be described in a modular way, as two separate and independent layers:

The Role Assignment Layer: Random anonymous committee members are selected, and RACCs are created. This takes place in the “*natural world*”, and is described as a protocol executed by real parties.

The Computation Layer: Computation runs on top of the RACCs. This takes place in the “*abstract world*”, and is described in terms of *roles* (i.e., “the j th member of the i th committee”). These roles are executed by the real parties selected for those roles in the role assignment layer.

One appealing property of the modular YOSO architecture is that it enables us to achieve security against an adaptive adversary given an MPC with *static* security. Canetti et al. [Can+23] show how role assignment can be used to compile a computation-layer-MPC that has static security in the abstract world into an MPC that has adaptive security in the natural world.

Previous approaches to YOSO role assignment suffered from one of three drawbacks:

1. Role assignment required broadcast complexity scaling linearly in the number N of participants [Cas+22];
2. In order to facilitate computation with guaranteed output delivery, role assignment required that significantly more than half (i.e., roughly three quarters) of the overall participants be honest [Ben+20; Gen+21a]; or
3. Role assignment relied on secure computation (implying either a complex correlated randomness setup assumption, which we don’t know how to instantiate with sublinear cost; or a circular dependency between role assignment and secure computation) [Gen+21b], [Dam+25].

Hence, the following question remained open:

Is there an end-to-end YOSO MPC protocol that has sublinear broadcast complexity, has (near) optimal resilience, and does not rely on complex correlated randomness setup assumptions?

⁵ Like previous works on the YOSO model, we assume atomic send, i.e., when a party multicasts to several other parties, corruption happens either before or after this. This is also known as a weak adaptive adversary.

Our Contributions. In this paper, we answer the above question in the affirmative by designing a YOSO MPC that bypasses *all* of the drawbacks above. Our protocol has polylogarithmic broadcast complexity, near optimal resilience, and relies on standard setup assumptions (PKI, bulletin board and random oracle), avoiding the need for correlated randomness.

We accomplish this by departing from the classic threshold adversary structure for committees. Guaranteed output delivery is only possible for $Q2$ adversary structures, where there do not exist two corruptible sets that cover the entire set of participants [Cle86]. All previous YOSO MPC protocols with guaranteed output delivery required that each committee contain a majority of honest parties, which is the most common $Q2$ adversary structure. In our protocol, each committee consists of m subcommittees, each of size m ; we require that a *majority of the committees contain a majority of honest parties*. This is a $Q2$ adversary structure which we refer to as the *nested* adversary structure. Notably, it allows the adversary to corrupt almost three fourths of the overall committee members, as long as $\frac{2}{3}$ of these corrupt committee members reside in a minority of the subcommittees.

As our first contribution, we generalize the role assignment ideal functionality of Canetti et al. [Can+23] to support more expressive adversary structures.

Second, we realize this more general ideal functionality by describing a communication-efficient, setup-free role assignment protocol that assumes a threshold adversary corrupting only a fraction $\frac{1}{2} - \epsilon$ of the natural-world participants, where ϵ is a constant. Our role assignment protocol requires a random beacon, a verifiable random function (VRF, [MRV99]), and key and message non-committing encryption (KMNCE, [Hua+22]). With overwhelming probability, this role assignment protocol creates committees satisfying the nested adversary structure described above, in the abstract world⁶.

As our third contribution, using an abstraction of known techniques from [Dam+25] and [BDO23], we show how to run efficient MPC on top of exactly such committees, using a nested form of integer secret sharing [DT06] coupled with CDN MPC [CDN01], which leverages linearly homomorphic encryption with threshold decryption.

Finally, as a complementary contribution, we prove that *every* MPC with B broadcast complexity must rely on $\frac{N}{2} + o(\frac{N}{B})$ honest parties or have constant error probability. Taking B to be polylogarithmic (in N) indicates that we would need $\frac{N}{2} + o(\frac{N}{\text{polylog}(N)})$ honest parties, implying that our construction is near-optimal in terms of the number of corruptions it can tolerate.

⁶ We note that our result is incomparable to the recent role assignment protocol of Damgård et al. [Dam+25]. The protocol of Damgård *et. al* has the advantages of adaptive security, running on an asynchronous network, and avoiding the use of KMNCE. On the other hand, it has the *disadvantages* of requiring an elaborate setup, and having a circular dependency between the role assignment and the MPC.

1.1 Technical Overview: Upper Bounds

In this subsection, we give a high-level overview of our role assignment and secure computation constructions⁷. In Section 1.1.1, we describe the role assignment protocol, which is run in the natural world by all N participants. (Even though all N participants are involved, the protocol remains communication-efficient because only a small subset of the participants send messages.) In Section 1.1.2, we describe the secure computation protocol, which is run in the abstract world by a sequence of committees.

In Figure 1, we give a visual summary of our constructions and their dependencies on cryptographic primitives. In terms of setup, our constructions require a PKI (the availability of all parties' public keys), a public bulletin board, and a random oracle.

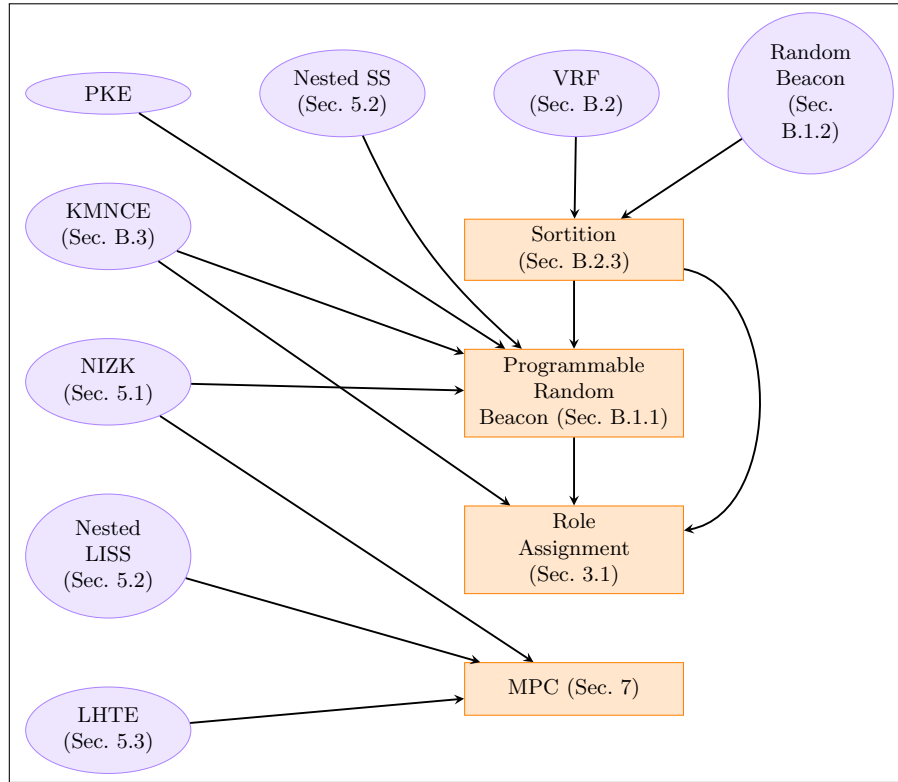


Fig. 1. A map of our constructions. Protocols appear in orange; building blocks appear in purple.

⁷ Our construction uses broadcast as a black box; for one possible implementation of broadcast in our setting, see Dettling et al. [Det+25].

1.1.1 Role Assignment We want to assign roles to which secret messages can be sent, or, in other words, *form a secret-holding committee*. In our protocol, a committee of nominators self-elect, and each nominator then nominates a subcommittee of secret holders. The idea of nominators selecting secret holders was introduced by Benhamouda et al. [Ben+20], and used by [Gen+21a]; the idea of a single nominator selecting a set of parties — rather than a single party — was also seen in the work of Abraham et al. [Abr+25] in the context of asynchronous consensus.

Let m be a number large enough such that when m parties are sampled at random from the set of all N participants, at least half are honest with overwhelming probability. Our role assignment protocol proceeds as follows:

1. m *nominators* self-elect using a VRF-based Algorand-style cryptographic sortition. As in the work of Benhamouda et al. [Ben+20] and Canetti et al. [Can+23], the nominators cannot form a secret-holding committee directly, since someone who might want to send them a secret has no way to efficiently do so (without relying on, e.g., witness encryption); instead, the nominators continue the protocol as follows.
2. Each nominator i does the following:
 - (a) Chooses m *parties* to serve on a secret-holding subcommittee. (This is in contrast to the works of Benhamouda *et. al* and Canetti *et. al*, where each nominator chooses only a single party.)
 - (b) For each $j \in [1, \dots, m]$:
 - i. Samples an “ephemeral” public key encryption key-pair $(ek_{i,j}, dk_{i,j})$.
 - ii. Encrypts $dk_{i,j}$ to the j th member of the secret-holding subcommittee using key-message non-committing encryption (KMNCE), as done by Canetti *et. al* [Can+23, Section 7]. (KMNCE automatically provides anonymity, such that by looking at a ciphertext it is impossible to tell who the intended recipient is.) Let $c_{i,j}$ denote the resulting ciphertext.
 - (c) Erases all of its secret state.
 - (d) Broadcasts $\{(ek_{i,j}, c_{i,j})\}_{j \in [1, \dots, m]}$.
3. A random permutation (derived from a random beacon) is applied to the subcommittees, and to the roles on each subcommittee.
4. Each party uses its long-term KMNCE secret key to test whether it is on the secret-holding committee, and to retrieve its new ephemeral secret key dk if so.

While corrupt nominators might choose subcommittees that are themselves entirely corrupt, honest nominators will choose subcommittees which are *mostly* honest, as a consequence of our choice of the parameter m . Therefore, we will end up with a secret-holding committee of subcommittees most of which are mostly honest, satisfying our nested adversary structure in the abstract world.

There are two mechanics (inherited from Canetti et al. [Can+23]) in the above role assignment protocol which enable its use to compile abstract-world MPC protocols with static security into natural-world MPC protocols with adaptive security. The first is the use of KMNCE, which lets the simulator handle

adaptive corruptions of honestly-nominated holding committee members. The simulator creates some “fake” ciphertexts on behalf of honest nominators, which enables it to decide who a ciphertext is intended for later. The simulator uses this to dynamically assign statically-corrupted roles to adaptively-corrupted parties.

The second mechanic is the use of the permutation in step (3) above; this gives the simulator control over *corrupt* nominators’ nominations. A corrupt nominator could e.g. time its nomination so as to target a specific subcommittee. Since role corruptions are static, the simulator cannot let a corrupt nominator determine which subcommittee is mostly corrupt. The simulator programs the random beacon (which determines the permutation applied to the nominated committee) to take back control. To this end, it is crucial that the beacon remain unpredictable up until the point where all nominations have been made. See Section B.1.1 for a discussion on how we instantiate our beacon.

1.1.2 Secure Computation We do MPC by leveraging linearly homomorphic encryption with threshold decryption, as done by Cramer, Damgård and Nielsen (CDN, [CDN01]). The CDN approach was made YOSO by Gentry et al. [Gen+21a], and was further developed by Braun, Damgård, and Orlandi [BDO23] to include class-group-based encryption and by Damgård et al. [Dam+25] to include role-assignment and more efficient resharing. In these constructions, a single private/public key pair is associated with the system. Inputs to the computation are encrypted to the public key, and the corresponding private key is secret-shared.

In our case, the private key must be shared according to a nested access structure. To this end, we introduce a nested form of integer secret sharing. Instead of basing our nested integer secret sharing on Shamir’s scheme [Sha79], we use a more general form of linear integer secret sharing due to Damgård and Thorbek [DT06], where the sharing matrix may consist only of $-1, 0, 1$ entries. The linear properties of this secret sharing scheme facilitate both one-round decryption and one-round key re-sharing to the next committee.

In some prior work [Gen+21a], integer re-sharing resulted in dramatic share-size increase, due mostly to tricks used to address the fact that Lagrange coefficients (for Shamir sharing) can be non-integers. While the scheme we use does not have this problem, any integer secret sharing scheme must have shares significantly larger than the secret. So, naive resharing would still result in a slow but steady share growth.

We circumvent this issue with a trick borrowed from Damgård et al. [Dam+25]: we provide committees i and $i + 1$ with two independent sharings of the same random value ρ . Committee i publicly reconstructs the secret key masked additively with ρ ; committee $i + 1$ derives fresh shares of the secret key by subtracting their shares of ρ from the reconstructed value.

As in the previous CDN-based constructions, addition is performed locally over ciphertexts, and multiplication is accomplished with the aid of Beaver triples (generated by a few self-elected, dishonest-majority committees). One advantage of using the CDN approach is that values encrypted to the system public key

are implicitly passed from one committee-of-committees to the next, as the decryption key is re-shared.

1.2 Technical Overview: Lower Bounds

In order to establish a bound between low communication and protocol resilience we follow the standard approach of assuming such a protocol exists and deriving a contradiction via a set of worlds with indistinguishable views. Our lower bound holds in the multicast model where honest parties are only allowed to send multicasts during the protocol. Note that YOSO protocols are indeed multicast protocols.

One such bound by Rambaud [Ram23] showed that every consensus protocol in the multicast model requires a linear number of multicasts if it has the optimal resilience of $N = 2t + 1$. To strengthen the bound, we consider consensus with only (up to t) omission faults⁸.

We extend this lower bound to additional natural notions of consensus, such as consensus with external validity [CKS05]. We consider uniform consensus protocols, in which all parties that output values must agree, even if some of them are faulty. In the Byzantine setting this is analogous to assuming that the output of the consensus protocol is provable to external entities.

At a high level, the lower bound follows from splitting the parties into two nearly equal sets and blocking communication between the two sets. The main technical challenge is showing that even though one set is a majority, its size is so close to half that if only few parties multicast, then their view is indistinguishable from the case that they are the minority. This will allow us to execute a “split-brain” attack similar to that of Dwork, Lynch, and Stockmeyer [DLS84], by having two halves of the network not communicate with each other.

1.3 Paper Outline

In Appendix A, we briefly recall the definition of YOSO-security, which we will use for both our role assignment and our MPC protocols.

In Section 2, we generalize the role assignment ideal functionality from the work of Canetti et al. [Can+23]. In Section 3, we describe the protocol itself (and in Appendix B, we describe the building blocks it uses). The proof appears in Appendix C.

In Section 4, we briefly describe how to compile a statically secure protocol in the abstract world to an adaptively secure protocol in the natural world.

In Section 5, we describe the building blocks for our secure computation protocol, and in Sections 6 and 7, we describe the protocol itself.

Finally, in Appendix D we describe our lower bound.

⁸ Note that security against a weaker adversary makes for a weaker construction, but impossibility of security against a weaker adversary makes for a stronger lower bound.

2 General Role Assignment Functionality

In this section we present the ideal functionality $\mathcal{F}_{\text{RA-general}}$, a generalization of the functionality $\mathcal{F}_{\text{RA}}$ from the work of Canetti *et. al* [Can+23]. This functionality assigns parties to roles while keeping this mapping hidden.

While $\mathcal{F}_{\text{RA}}$ considered a sequence of committees each of which should have an honest majority, we instead consider a general adversary structure A which should be satisfied within each committee. We use the following notation:

- Let each committee consist of n roles.
- Let ccor denote a list of n bits, representing which of the roles in a given committee are corrupt.
- We let $A(\text{ccor}) = \text{allowed}$ if the corruptions that ccor represents are allowed according to the adversary structure A . Otherwise, $A(\text{ccor}) = \text{disallowed}$.

In addition to A , we parametrize our functionality with a set of permutations PERMS . It is important that for any $\pi \in \text{PERMS}$ and for any list L of n bits, $A(L) = A(\pi(L))$.

Remark 1 (On Permuting Roles). In $\mathcal{F}_{\text{RA-general}}$, as in $\mathcal{F}_{\text{RA}}$, before the functionality finalizes a committee, it permutes the roles. This is important to prevent the adversary from targeting a specific role for corruption; if we allow the adversary to do this, we will not be able to achieve security against an adaptive adversary by compiling a protocol which only offers security against a *static* adversary in the abstract world.

Remark 2 (On Binding Nominations to Roles). In $\mathcal{F}_{\text{RA}}$, nominations were not bound to roles, because all roles within a committee were equivalent. In $\mathcal{F}_{\text{RA-general}}$, nominations *are* bound to specific roles. This is to accommodate the asymmetry of trust in general adversary structures.

The functionality permutes the roles during committee finalization, so the roles for which parties are nominated are not necessarily the roles they end up with; however, some characteristics are persevered, so it is still important to allow targetted nomination. (E.g., in the case of our nested adversary structure, permutations are subcommittee-preserving, so two roles within the same subcommittee always remain within the same subcommittee.)

Finally, while nominations are bound to specific roles, (adaptive) corruptions are not.

We define a key generation function $\text{Gen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$, where pk contains all public information associated with a given role, and sk contains all secret information for that role. In $\mathcal{F}_{\text{RA}}$, Gen was fixed to generate a (public key) encryption key pair and a signature key pair.

Functionality $\mathcal{F}_{\text{RA-general}}(\mathcal{P}, n, A, \text{PERMS}, \mathcal{D}, \text{delay})$:

This functionality is synchronous, namely it has access to global clock functionality as in the model of Katz *et. al* [Kat+13]. It has the following parameters:

- $\mathcal{P}$: the set of parties.
- n : the size of a committee.
- A : the adversary structure.
- $\mathcal{D}$: a sampling algorithm, and
- **delay**: the upper bound on the number of rounds required to complete nomination.

The functionality is notified by the adversary whenever a party is corrupted or restored, and maintains the current partition of $\mathcal{P}$ into the sets $\mathcal{H}$ and $\mathcal{I}$ of all honest and corrupt party identifiers, respectively. It also maintains a global set $\mathcal{R}$ of ready parties, initially equal to $\mathcal{P}$.

New committee: On input (New, cid) from all honest parties up until the round r specified by cid, store

$$(\text{cid}, \text{pkeys} = [\perp, \dots, \perp], \text{skeys} = [\perp, \dots, \perp], \text{ccor} = [0, \dots, 0], \text{finished} = \perp).$$

Ignore the command if any value is already stored for cid.

- The lists **pkeys** and **skeys** each initially contain n copies of $\perp$. The list **pkeys** will be updated with **pk**, where **pk** in position R refers to the public key established for role R . The list **skeys** will be updated with tuples (pid, sk), where (pid, sk) in position R refers to the secret key corresponding to the role R which has been assigned to party with identifier pid.
- The list **ccor** initially contains n copies of 0, indicating that no member of the committee is corrupt. Later, a 1 in position R refers to the corruption of role R .

Each time an honest party inputs (New, cid), forward this to the simulator $\mathcal{S}$.

Nominate honest: On input (NomHonest, cid, R) from the simulator $\mathcal{S}$, retrieve the value (cid, pkeys, skeys, ccor, finished).

- If no such value exists, if finished = $\top$ or if **pkeys**[R] $\neq \perp$, do nothing.
- Generate fresh keys for the chosen party:

$$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda).$$

- Set **pkeys**[R] = **pk**, and set **skeys**[R] = ($\perp$, **sk**).
- Output (NomHonest, cid, R , **pk**) to the simulator $\mathcal{S}$.

Nominate corrupt: On input (NomCorrupt, cid, R , pid, **pk**, **sk**) from the simulator $\mathcal{S}$, retrieve the value (cid, pkeys, skeys, ccor, finished).

- If no such value exists, if finished = $\top$ or if **pkeys**[R] $\neq \perp$, do nothing.

- Let ccor' be ccor modified to have a 1 in position R .
 - If $A(\text{ccor}') = \text{disallowed}$, do nothing.
 - Else, let $\text{ccor} = \text{ccor}'$ (the invariant that $A(\text{ccor}) = \text{allowed}$ is maintained).
- Set $\text{pkeys}[R] = \text{pk}$, and set $\text{skeys}[R] = (\text{pid}, \text{sk})$.

Corrupt nominee: On input $(\text{CorruptNominee}, \text{cid}, \text{pid})$ from the simulator $\mathcal{S}$, retrieve the value $(\text{cid}, \text{pkeys}, \text{skeys}, \text{ccor}, \text{finished})$.

- Let R be a random role such that (a) the role has already been nominated (i.e., $\text{pkeys}[R] \neq \perp$), (b) the role is honest (i.e., $\text{ccor}[R] = 0$), and (c) the role is corruptible (i.e., when ccor is modified to have a 1 in position R , $A(\text{ccor}') = \text{allowed}$). If no such R exists, do nothing.
- Modify ccor as above.
- Retrieve $(\perp, \text{sk})$ from $\text{skeys}[R]$.
- Set $\text{skeys}[R] = (\text{pid}, \text{sk})$.
- Output $(\text{CorruptNominee}, \text{cid}, R, \text{pid}, \text{sk})$ to the simulator $\mathcal{S}$.

Finish: On input $(\text{Finish}, \text{cid})$ from all honest parties, retrieve the value $(\text{cid}, \text{pkeys}, \text{skeys}, \text{ccor}, \text{finished})$.

- Let r be the current round (read from the global clock), and r' be the round number encoded in cid . If $r' + \text{delay} \neq r$, do nothing.
- If pkeys contains any $\perp$ values or if $\text{finished} = \top$, do nothing.
- Sample a random permutation $\pi \leftarrow \text{PERMS}$.
- Apply π to pkeys , skeys and ccor .
- For each $R \in [n]$, if $\text{skeys}[R]$ is of the form $(\perp, \text{sk})$, choose an honest party according to our sampling algorithm, as $\text{pid} \leftarrow \mathcal{D}(\mathcal{H})$. If $\text{pid} \in \mathcal{R}$ (that is, if the chosen party is ready), set $\text{skeys}[R] = (\text{pid}, \text{sk})$.
- Set $\text{finished} = \top$ for cid .

Output $(\text{Finish}, \text{cid}, \pi, \text{pkeys})$ to the simulator $\mathcal{S}$.

Read (used to learn assigned roles): On input $(\text{Read}, \text{cid})$ from party P_{pid} , give it all of the public keys, and the secret keys corresponding to pid . More formally, retrieve the value $(\text{cid}, \text{pkeys}, \text{skeys}, \text{ccor}, \text{finished})$. If no such value exists, if $\text{finished} = \perp$ or if P_{pid} has read cid before, do nothing.

- For every role R , let

$$\text{skeys}_{\text{pid}}[R] := \begin{cases} \text{skeys}[R] & \text{if } \text{skeys}[R] = (\text{pid}, \cdot) \\ \perp & \text{else} \end{cases}$$
- Output $(\text{pkeys}, \text{skeys}_{\text{pid}})$ to P_{pid} .

Delete (used just prior to execution of assigned roles): On input (Delete) from party P_{pid} , do the following:

- Overwrite all elements of skeys of the form (pid, sk) with $(\text{pid}, \perp)$. Disallow any future signing queries by P_{pid} .
- Set $\mathcal{R} \leftarrow \mathcal{R} \setminus \{\text{pid}\}$.
- Output $(\text{Delete}, \text{pid})$ to $\mathcal{S}$.

Ready: On input (Ready) from party P_{pid} , update the global ready set $\mathcal{R} \leftarrow \mathcal{R} \cup \{\text{pid}\}$ in the beginning of the subsequent round. Output (Ready, pid) to the simulator $\mathcal{S}$.

Corrupt: Upon receiving (Corrupt, pid*) from $\mathcal{Z}$, output (Corrupt, pid*, skeys_{pid*}) to $\mathcal{S}$ (where skeys_{pid*} is defined as in **Read**).

3 Role Assignment Protocol

In this section, we describe a role assignment protocol that realizes our functionality for the nested adversary structure. The building blocks (a random beacon, a verifiable random function (VRF) and a key-message non-committing encryption (KMNCE) scheme) are described in Appendix B. In this section, we use the random beacon as an ideal functionality; however, in Appendix B.1, we describe how two types of random beacon — an (unpredictable) random beacon, and a programmable random beacon — can be instantiated in our setting.

We use two related parameters: m and t . m should be set in such a way that, when choosing m random participants from our total pool of N parties, the m chosen participants have fewer than a fraction t of corrupt parties with overwhelming probability. (So, m is actually a function of N and t .) We omit the exact analysis of how m should be set, and instead refer the reader to the work of Benhamouda *et. al* [Ben+20].

3.1 Protocol

We describe a role assignment protocol for committees of size $n = m^2$, with adversary structure $A_{\text{nested}}^{m,t}$ for a fraction $0 < t < 1$.

In order to describe $A_{\text{nested}}^{m,t}$ intuitively, we index the corruption indicator list ccor of size $n = m^2$ by pairs of indices $(i, j) = im + j$, denoting the index of the j th member of the i th subcommittee. We define

$$A_{\text{nested}}^{m,t}(\text{ccor}) = \begin{cases} \text{disallowed} & \text{if } \begin{array}{l} \text{there exist } i_1, \dots, i_{\lceil tm \rceil} \text{ such that for each } i_j, \\ \text{there exist } j_1, \dots, j_{\lceil tm \rceil} \text{ such that } \text{ccor}[(i_j, j_1) \\ = \dots = \text{ccor}[(i_j, j_{\lceil tm \rceil})] = 1 \end{array} \\ \text{allowed} & \text{else} \end{cases}$$

$\text{PERMS}_{\text{nested}}^m$ is defined to be the family of permutations which rearrange each subcommittee internally, and the subcommittees among themselves (however, each subcommittee remains a subcommittee composed of the same elements). Crucially, it holds that for any such permutation $\pi \in \text{PERMS}_{\text{nested}}^m$ and any $L \in \{0, 1\}^n$, $A_{\text{nested}}^{m,t}(L) = A_{\text{nested}}^{m,t}(\pi(L))$.

Remark 3. In the functionality, each nomination is for a specific role. In the construction, this is not the case because a nominator cannot win the right to nominate a specific sub-committee. Winning the right to nominate sub-committee j

for a specific j would mean that the sortition should guarantee a *single* winner; this is hard to guarantee with non-negligible probability. This is not problematic in the construction because the adversary structure $A_{\text{nested}}^{m,t}$ is symmetric with respect to all subcommittees.

We let p be the probability with which any given party should be a member of the nominating committee; it should hold that $m = pN$, where N is the total number of participants.

Our protocol takes two rounds (ignoring additional rounds it may take to generate a beacon value), due to our need for two sequential random beacon values. The committee is nominated in round r , where the nominators use the r th beacon value for the sortition. It is then read in round $r+1$, where everyone uses the $(r+1)$ st beacon value to define a permutation on the nominated committee.

Protocol $\Pi_{\text{RA-nested}}(\mathcal{P}, n = m^2, t)$

Everyone has access to:

- A random beacon
- A PKI containing the following for each party P :
 - A KMNCE public encryption key, denoted $P.\text{ek}$
 - A VRF verification key, denoted $P.\text{vk}$

Each party P additionally stores the following:

- Its current long-term KMNCE secret decryption key as $P.\text{dk}$
- Its secret VRF evaluation key $P.\text{sk}$

Each party generates their own VRF and KMNCE keys.

New Committee: On input (New, cid) in round r (where r is implicitly encoded in cid), party P_{pid} does the following:

1. If this is not the first **New** command with committee identifier cid, ignore this command.
2. Input (Read, r) to the random beacon, to receive random ρ .
3. Compute $(\text{draw}, \pi) \leftarrow \text{VRF.Eval}(P_{\text{pid}}.\text{sk}, (\rho, \text{cid}))$.
4. If draw is a winning draw (i.e. $\text{draw}/2^{\ell_{\text{VRF}}} \leq p$), proceed to nominate m parties. Otherwise skip the remaining steps.
5. For each $j \in [m]$:
 - (a) Sample a uniformly random party $\text{pid}_j \leftarrow \mathcal{P}$.
 - (b) Generate fresh ephemeral keys for the nominated role:

$$(\text{pk}_j, \text{sk}_j) \leftarrow \text{Gen}(1^\lambda).$$

- (c) Encrypt the secret key to the chosen machine:

$$c_j \leftarrow \text{KMNCE.Enc}(P_{\text{pid}_j}.\text{ek}, (\text{pid}_j, \text{sk}_j)).$$

- (d) Erase the key sk_j , all randomness used for sampling the keys, pid_j , as well as any encryption randomness.

6. Post

$$\left(\text{cid}, \text{pid}, \text{draw}, \pi, (\text{pk}_1, c_1), \dots, (\text{pk}_m, c_m) \right)$$

to the bulletin board.

Read: On input $(\text{Read}, \text{cid})$ in round r , party P_{pid} finds the round r' encoded in cid . If $r \geq r' + 1$, party P_{pid} does the following:

1. Input (Read, r') to the random beacon to receive randomness $\rho_{r'}$.
Input $(\text{Read}, r' + 1)$ to the random beacon to receive randomness $\rho_{r'+1}$.
2. Retrieve from the bulletin board a list of messages for committee identifier cid posted in round r' :

$$\left(\text{cid}, \text{pid}_1, \text{draw}_1, \pi_1, (\text{pk}_{1,1}, c_{1,1}), \dots, (\text{pk}_{m,1}, c_{m,1}) \right),$$

$\dots,$

$$\left(\text{cid}, \text{pid}_{m'}, \text{draw}_{m'}, \pi_{m'}, (\text{pk}_{m',1}, c_{m',1}), \dots, (\text{pk}_{m',m}, c_{m',m}) \right)$$

3. Remove any messages from the list where draw_i is not a winning draw. This may be verified by running $\text{VRF.Ver}(P_{\text{pid}}.\text{vk}, (\rho_{r'}, \text{cid}), \text{draw}_i, \pi_i)$. Remove the element if VRF.Ver returns **reject**, or if $\frac{\text{draw}_i}{2^{\ell_{\text{VRF}}}} > p$.
4. Sort the list lexicographically by draw , keeping only the m first elements. If the list does not have at least m elements, pad it with values $(\text{cid}, \perp, \dots, \perp)$.
5. Let $\pi \leftarrow \text{PERMS}_{\text{nested}}^m$ be a random allowable permutation on $[n]$ defined by the randomness $\rho_{r'+1}$. Apply π to the list.
6. For each element $\left(\text{cid}, \text{pid}_i, \text{draw}_i, \pi_i, (\text{pk}_{i,1}, c_{i,1}), \dots, (\text{pk}_{i,m}, c_{i,m}) \right)$ in the list, for each $j \in [m]$:
 - (a) Append $(\text{pk}_{i,j}, c_{i,j})$ to pkeys .
 - (b) Attempt to decrypt $(\text{pid}, \text{sk}) \leftarrow \text{KMNCE.Dec}(P_{\text{pid}}.\text{dk}, c_{i,j})$.
 - i. If $(\text{pid}, \text{sk}) \neq \perp$ and pid_i matches the identifier of the party which posted the element to the bulletin board, set $\text{skeys}[(i, j)] = (\text{pid}, \text{sk})$.
 - ii. Else, set $\text{skeys}[(i, j)] = (\perp, \perp)$.
7. Store $(r, \text{cid}, \text{pkeys}, \text{skeys})$.

Delete: When given input **Delete**, for each stored value $(r, \text{cid}, \text{pkeys}, \text{skeys})$, delete skeys , overwriting it with the empty list. Finally, delete the long term secret key $P.\text{sk}$.

Ready: When given input **Ready**, generate a new key pair $(\text{ek}, \text{dk}, \text{tk}) \leftarrow \text{KMNCE.Gen}(1^\lambda)$, setting $P.\text{sk} = \text{sk}$ and deleting tk immediately. Post (pid, ek) to the bulletin board.

Let $\mathcal{F}_{\text{RA-nested}}(\mathcal{P}, n = m^2, t)$ refer to $\mathcal{F}_{\text{RA-general}}(\mathcal{P}, n = m^2, A_{\text{nested}}^{m,t}, \text{PERMS}_{\text{nested}}^m, \mathcal{D}, \text{delay} = 1)$ for the uniform sampling function $\mathcal{D}$, and for $t = \frac{|\mathcal{X}|}{|\mathcal{P}|} + \epsilon(|\mathcal{P}|)$ (for a negligible function ϵ).

Theorem 1.

$$\Pi_{\text{RA-nested}}(\mathcal{P}, n = m^2, t)$$

UC-realizes

$$\mathcal{F}_{\text{RA-nested}}(\mathcal{P}, n = m^2, t).$$

We defer the proof of Theorem 1 to Appendix C.

4 Compiling Abstract to Natural

We can compile protocols with static security in the *abstract world* to protocols with adaptive security in the *natural world* in the same way as done by Canetti *et. al* [Can+23, Section 5]. We omit details here, and instead summarize a high-level intuition of the compiler of Canetti *et. al*. The compiler uses $\mathcal{F}_{\text{RA-nested}}$ with a key generation function that produces encryption and signature keys.

Compiler Execution. In the compiled protocol in the natural world, the parties have access to a public bulletin board and to $\mathcal{F}_{\text{RA-nested}}(\mathcal{P}, n = m^2, t = \frac{1}{2})$. Each party invokes $\mathcal{F}_{\text{RA-nested}}$ to determine whether it has been selected for a role. If it has been selected for a role, it learns that role's secret (decryption and signing) keys, which we take to be a signing and decryption key. It can then read messages encrypted to its role from the public bulletin board, verify the signatures attached to those messages, and decrypt them. The party then executes its role, and obtains the role's outgoing messages. It (1) encrypts those outgoing message to future roles' encryption keys (again provided by $\mathcal{F}_{\text{RA-nested}}$), (2) signs those messages with the role's signing key, (3) deletes all its locally-stored secrets (and sends **Delete** to $\mathcal{F}_{\text{RA-nested}}$), and finally (4) posts the signed ciphertexts to the public bulletin board, for subsequent roles to retrieve.

Compiler Security (Informal). In order to prove that the compiled protocol is secure against an adaptive adversary $\mathcal{A}_{\text{nat}}$ in the natural world, we must describe a simulator $\mathcal{S}_{\text{nat}}$ for such an adversary. We assume the original protocol is secure against a static adversary in the abstract world, which implies the existence of a simulator $\mathcal{S}_{\text{abs}}$ for it.

Our simulator $\mathcal{S}_{\text{nat}}$ will choose a maximal random set of abstract-world roles satisfying the nested adversary structure to statically corrupt. $\mathcal{S}_{\text{nat}}$ then needs to do two things when it runs $\mathcal{A}_{\text{nat}}$: (a) program $\mathcal{F}_{\text{RA-nested}}$ to make sure corrupt natural-world parties are only assigned the statically corrupt abstract-world roles, and (b) produce messages on behalf of honest natural-world parties.

To accomplish (a), we need to consider two events: (a1) nominations of corrupt natural-world parties (either by other corrupt natural-world parties or by honest natural-world parties, via **NomCorrupt** commands to $\mathcal{F}_{\text{RA-nested}}$), and (a2)

corruptions of nominated parties (via `CorruptNominee` commands to $\mathcal{F}_{\text{RA-nested}}$). To take care of (a1), $\mathcal{S}_{\text{nat}}$ manipulates the random beacon to dictate a permutation π that maps all these nominations to statically corrupt abstract-world roles. To take care of (a2), when $\mathcal{A}_{\text{nat}}$ corrupts a natural-world party, with the appropriate probability, $\mathcal{S}_{\text{nat}}$ assigns a remaining corrupt abstract-world role to that party. Otherwise, it assigns no roles. (While the real $\mathcal{F}_{\text{RA-nested}}$ assigns roles to parties during nomination, not upon corruption, this process is hidden from $\mathcal{A}_{\text{nat}}$ so the adversary will not be able to distinguish the two.)

If $\mathcal{A}_{\text{nat}}$ issues a `Read` command on behalf of a natural-world party, if any abstract-world roles have been assigned to that party, $\mathcal{S}_{\text{nat}}$ returns those roles' secret information to $\mathcal{A}_{\text{nat}}$. $\mathcal{A}_{\text{nat}}$ may then post those roles' outgoing messages on the public bulletin board.

To accomplish (b), $\mathcal{S}_{\text{nat}}$ assigns the remaining abstract-world roles to honest natural-world parties. On behalf of unclaimed corrupt roles, it follows the protocol honestly; on behalf of honest roles, it runs $\mathcal{S}_{\text{abs}}$ to produce messages.

5 Building Blocks for MPC

In this section we outline the primitives we will need for our YOSO MPC protocol. Since we are mainly re-purposing material from previous work, we do not formalize everything, but refer in-line to previous papers for details. We emphasize that the encryption scheme we use in this part of the paper is different from the (non-committing) one we used for implementing the role assignment functionality.

We also emphasize that in this and the following two sections on subprotocols and MPC, we are in the “abstract” world, where the protocols are executed by roles, each of which speaks only once. Nevertheless, we will refer to these roles as parties, to be more in line with the usual lingo used for specifying protocols. It should be understood throughout that the roles will eventually be assigned to “real” parties using the role assignment from Section 3.1.

Our set of parties (roles) will be divided into a number of (disjoint) committees, each of which has $n = m^2$ members, split into m subcommittees of size m , as described previously. In this section, we assume static corruptions, where the adversary is able to corrupt at most an unqualified subset of each committee; i.e., in each committee, a majority of the subcommittees have honest majority. The compiler from Section 4 converts security against static corruptions in the abstract world to security against adaptive corruptions in the natural world.

5.1 Zero-Knowledge

We will require non-interactive zero-knowledge proofs of knowledge for a number of statements.

5.1.1 Syntax Let w be the prover's secret witness, x the public data (“statement”), and R the relation the prover claims is satisfied. A non-interactive zero-knowledge proof of knowledge consists of two algorithms:

$\text{NIZK}(x; w : R) \rightarrow \pi$: given the statement x and the witness w , this algorithm produces a proof π that $R(x, w)$ is satisfied.
 $\text{Ver}(x, \pi : R) \rightarrow \{\text{accept}, \text{reject}\}$: given the statement x and the proof π of knowledge of a witness w such that $R(x, w) = 1$, this algorithm checks the proof and either accepts or rejects it.

5.1.2 Properties We will require the proofs to be correct, simulation-sound and straight-line extractable.

Correctness: Any honestly-generated proof should be accepted by the Ver algorithm.

Zero Knowledge: A proof π should reveal nothing about w ; that is, there should exist a simulator (with access to a trapdoor td or with control over a programmable random oracle) which, given just the statement x and no witness w , can produce a proof indistinguishable from an honestly-generated one.

Simulation-soundness [Sah99]: A prover without knowledge of a witness w such that $R(x, w) = 1$ should be unable to come up with a verifying proof even given access to simulated proofs of its choosing.

Straight-line Extractability: There should exist an extractor (with access to a trapdoor td or the queries made to an oracle) which can extract a valid witness w from a proof π .

5.1.3 Instantiations A non-interactive zero-knowledge proof of knowledge with the properties we want can be achieved using generic techniques, but for the concrete instances that we need, it can be done quite efficiently by constructing a Σ -protocols for the relation in question and then applying the Fishlin transform. For more details on concrete instantiations, see [Del+24] and [BDO23].

5.2 Linear Integer Secret Sharing

The material on linear integer secret sharing in this section is taken from [DT06].

We will need to secret share the secret decryption key from the linearly homomorphic encryption scheme described in Section 5.3. In all known instantiations of linearly homomorphic encryption, ciphertexts are group elements and decryption takes place by raising the ciphertext to a power equal to the key; so, the key “lives in the exponent”. The natural choice would therefore be a sharing scheme that uses arithmetic modulo the order of the group. However, the group order needs to be secret, so we must resort to secret-sharing over the integers instead. Some earlier work [Gen+21a] used integer variants of Shamir’s scheme, but this leads to technical difficulties (e.g., unsustainable share growth as a consequence of re-sharing) due to the fact that the Lagrange coefficients needed for interpolation are not integers.

We will instead use a linear integer sharing scheme (LISS), which allows us to avoid these problem and simplify the protocols, at the cost of a larger share size initially (but eventually smaller) .

Sharing. A LISS is defined by a public *sharing matrix* M with c columns, ℓ rows, and integer entries. (One can think of M as a generalization of the Van der Monde matrix from Shamir's scheme.)

A dealer can share a secret number $s \in [0, 2^b]$ for a publicly known bit-length b among n parties. To do this, the dealer chooses a column vector $\mathbf{v}_s$ with s as the first entry and sufficiently large random numbers in the other entries (we make this precise in a moment). Shares are computed as the product $M \cdot \mathbf{v}_s$. (This corresponds to evaluating the polynomial in a number of points in Shamir's scheme.)

Each row of M is labeled with an index $i \in [1, n]$, and is owned by one party. (Each party owns one *or more* rows.) Our notation for this is that row i is owned by party $u(i)$. The dealer hands the i th entry of $M \cdot \mathbf{v}_s$ to party $u(i)$. In a slight abuse of notation, we refer to an entry in $M \cdot \mathbf{v}_s$ as a *share*, though each party may receive several entries.

Reconstruction. For each set A of parties, we let M_A denote the matrix we obtain by selecting from M only the rows owned by parties in A . If A is qualified to reconstruct the secret, there exists a *reconstruction vector* $\mathbf{r}_A$ with the property that

$$\mathbf{r}_A \cdot (M_A \cdot \mathbf{v}_s) = s,$$

corresponding to interpolation in Shamir's scheme.

Privacy. If A is not qualified to reconstruct, there exists a *sweeping vector* $\mathbf{w}_A$, which has 1 as its first entry, and further has the property that $M_A \cdot \mathbf{w}_A$ is the all-0 vector. It can be shown that the existence of $\mathbf{w}_A$ implies statistical privacy of the scheme. Namely, we define $w_{\max}$ to be the maximal numeric value of any entry occurring in any $\mathbf{w}_A$. Then, a *valid sharing vector* for $s \in [0, 2^b]$ is a vector $\mathbf{v}_s$ with s in the first entry, and with the other entries chosen from $[0, 2^{b+\log_2(w_{\max})+\kappa}]$, where κ is the statistical security parameter. When we speak about valid sharing vectors in the following and the dealer distributing shares is honest, we will assume that the entries are chosen uniformly from the interval.

Lemma 1. *For any two secrets $s, s' \in [0, 2^b]$, the distributions of shares seen by A from sharing s or s' , with valid sharing vectors, are statistically indistinguishable.*

Proof. (Sketch) if s was shared using $\mathbf{v}_s$, one could instead have shared s' using $\mathbf{v}_s + (s' - s)\mathbf{w}_A$, and the parties in A would receive exactly the same shares. The numbers in $\mathbf{v}_s$ are a factor 2^κ larger than those in $(s' - s)\mathbf{w}_A$, so $\mathbf{v}_s + (s' - s)\mathbf{w}_A$ is statistically indistinguishable from a valid sharing vector for s' . The lemma follows.

It has been shown [DT06] that LISS schemes with polynomial share size exist for the threshold case, where any majority of the parties are qualified to reconstruct. This follows from the fact that polynomial-size monotone formulae exist for computing the majority function. We let M_{th} denote such a scheme for m parties. For this scheme, all entries in all sweeping vectors are 1 or -1 , so the numbers in a valid sharing vector just need to be κ bits longer than the secret.

5.2.1 Nested Sharing Using LISS In this paper, we will need to share a secret among m subcommittees each of size m . To do this, we first share the secret according to M_{th} , and we assign each resulting share to a subcommittee, thinking temporarily of the subcommittees as shareholders. Finally, each share is again shared among the members of the relevant subcommittee according to M_{th} .

It is not hard to see that this entire process can be phrased as one LISS scheme using a matrix M that we can build from M_{th} . We will not spell out the straightforward (but tedious) details of this. But essentially, M will be the tensor product of M_{th} with itself, and hence be a $(c^2 \times \ell^2)$ matrix.

In the scheme defined by this M , the sets qualified to reconstruct the secret will be any set $\mathcal{H}$ such that, for more than $m/2$ subcommittees C , $\mathcal{H} \cap C$ contains more than $m/2$ parties.

As for the parameter sizes, the shares produced by M_{th} have size $b + \kappa + c \log m$ bits, for a constant c since M_{th} has a number of columns that is polynomial in m . So, the sharing vector in the nested scheme needs to have entries that are κ bits longer, so this will be $b + 2\kappa + c \log m$. Then we form the final shares by adding at most polynomial in m of these entries, so the shares produced by M will be numbers of length $b + 2\kappa + 2c \log m$ bits.

5.2.2 VSS In this paper, we realize verifiable secret sharing (VSS) as a set of shares of a secret together with an encryption of each share under the public key of the share holder, and a zero-knowledge proof that the encryptions were correctly formed. We assume that the secret is to be shared among the members of a committee, where ek_j is the public encryption key of the j 'th member. Let s be the secret, $\mathbf{v}_s$ be a valid sharing vector, and let the shares be $(s_1, \dots, s_\ell) = M\mathbf{v}_s$. Then, we define the following data object:

$$\text{VSS}(s, \mathbf{v}_s, r_1, \dots, r_\ell) = (s_1, \text{Enc}_{\text{ek}_{u(1)}}(s_1; r_1) \dots, s_\ell, \text{Enc}_{\text{ek}_{u(\ell)}}(s_\ell; r_\ell))$$

This object is of course not directly verifiable, but when it is created, it will be accompanied by a zero-knowledge proof that it was correctly formed, making it publicly verifiable.

We will say that a committee *holds a VSS* if a valid data object as defined here has been created and s_i is known to (only) $P_{u(i)}$ from the committee.

Note that, by linearity of the secret-sharing and encryption schemes, we can do linear combinations of these objects by adding corresponding shares and multiplying corresponding ciphertexts to get a VSS of the corresponding linear combination of the secrets.

We write this as follows:

$$\text{VSS}(s, \mathbf{v}_s, r_1, \dots, r_\ell) + \text{VSS}(s', \mathbf{v}_{s'}, r'_1, \dots, r'_\ell) = \text{VSS}(s+s', \mathbf{v}_s + \mathbf{v}_{s'}, r_1+r'_1, \dots, r_\ell+r'_\ell).$$

Note that if a committee holds both VSS objects, this operation can be done locally by the committee members. Multiplication by a public integer can be done in a similar way.

Using non-interactive zero-knowledge (Section 5.1), we can use the tools just described to do non-interactive (one-round) and publicly verifiable VSS. The dealer D , with public encryption key ek_D , is assumed to hold an encryption of the secret to be shared, $c_s = \text{Enc}_{\text{ek}_D}(s)$. Based on this, she can form a VSS object for a committee with public keys $\text{ek}_1, \dots, \text{ek}_n$, and prove in zero-knowledge that it was correctly formed. Details are given in the **NonIntVSS** protocol.

Protocol NonIntVSS($b, \text{ek}_D, \text{dk}_D, c_s, \text{ek}_1, \dots, \text{ek}_n$)

Assumption: All parties know

- b , the bit size;
- ek_D , the encryption key of the dealer D ;
- $\text{ek}_1, \dots, \text{ek}_n$, the encryption keys of the receiving committee; and
- $c_s = \text{Enc}_{\text{ek}_D}(s)$, (where $s \in [0..2^b]$), an encryption of the secret s to be shared.

The dealer D holds dk_D , and the receiving committee members have secret keys $\text{dk}_1, \dots, \text{dk}_n$.

1. D decrypts c_s , chooses a valid sharing vector $\mathbf{v}_s$ and computes $(s_1, \dots, s_\ell) = M\mathbf{v}_s$.
2. For $i \in [1, \dots, \ell]$, D chooses randomness r_i and computes $c_i = \text{Enc}_{\text{ek}_{u(i)}}(s_i; r_i)$.
3. D computes $\pi = \text{NIZK}(b, \text{ek}_D, c_s, \text{ek}_1, \dots, \text{ek}_n, c_1, \dots, c_\ell; \text{dk}_D, \mathbf{v}_s, r_1, \dots, r_\ell; \text{Rel})$, where Rel denotes the following relation: $s = \text{Dec}_{\text{dk}_D}(c_s)$ satisfies $s \in [0..2^b]$, $\mathbf{v}_s$ is a valid sharing vector for s , $c_i = \text{Enc}_{\text{ek}_{u(i)}}(s_i; r_i)$ for $i = 1 \dots \ell$ where $(s_1, \dots, s_\ell) = M\mathbf{v}_s$.
4. D outputs $\text{NonIntVSS}(b, \text{ek}_D, \text{dk}_D, c_s) = (c_1, \dots, c_\ell, \pi)$.
5. Once the members of the receiving committee are given $\text{NonIntVSS}(b, \text{ek}_D, \text{dk}_D, c_s)$, with valid π , they decrypt their shares locally, and the committee now holds $\text{VSS}(s, \mathbf{v}_s, r_1, \dots, r_\ell)$.

In following, when we do not need to refer to the randomness for the encryptions, we will suppress it and just say that the committee holds $\text{VSS}(s, \mathbf{v}_s)$.

5.3 Linearly Homomorphic Encryption

$\text{Gen}, \text{Enc}, \text{Dec}$ will denote key generation, encryption and decryption algorithms for a homomorphic cryptosystem. Running $\text{Gen}(1^\lambda)$ outputs a key pair (ek, dk) . The plaintext space is assumed to be Z_M for a modulus M , where the choice of M depends on the instantiation, more on this below. Encryption of plaintext x using randomness r is denoted by $\text{Enc}_{\text{ek}}(x; r)$, where r is usually an integer chosen uniformly from an interval, see instantiations below for more on this. We naturally assume that decryption works, that is, we have $\text{Dec}_{\text{dk}}(\text{Enc}_{\text{ek}}(x; r)) = x$.

We assume the encryption function is linearly homomorphic, that is, ciphertexts are elements in a finite group and we have

$$\text{Enc}_{\text{ek}}(x; r) \cdot \text{Enc}_{\text{ek}}(x'; r') = \text{Enc}_{\text{ek}}(x + x' \bmod M; r + r')$$

Sometimes in the following, we will suppress the randomness from the notation and write $\text{Enc}_{\text{ek}}(x)$ for an encryption of x .

This type of encryption scheme will be used in two places: there will be global key ek that is used for encrypting the state of the secure computation, and we use M to denote the modulus for this scheme. In addition, each individual party P_i will be assigned a key ek_i , the modulus for this scheme will be called M_{ind} .

We will assume that our cryptosystem is CPA secure in the usual well-known sense.

5.3.1 Threshold Decryption We will need a version of the cryptosystem where distributed decryption can be done. For this purpose, we assume an extended version of the key generation, ThGen , which runs Gen and then secret shares the decryption key as described above, outputting $(\text{ek}, s_1, \dots, s_n)$ where ek is the public encryption key and the s_i 's are the shares of the decryption key.

A party holding a share s_i of the decryption key can contribute to decryption of a ciphertext c by running a partial decryption algorithm

$$\text{PartDec}_{s_i}(c) = d_i$$

where d_i is called a *decryption share*. Then, given decryption shares from a qualified set of parties, one can reconstruct the message contained in the ciphertext, using the plaintext reconstruction algorithm PlainTxtRec . Concretely, we require that, for any qualified set $\mathcal{H}$, and ciphertext $c = \text{Enc}_{\text{ek}}(x)$,

$$x = \text{PlainTxtRec}(\{\text{PartDec}_{s_i}(c) \mid i \in \mathcal{H}\}).$$

5.3.2 Security We want distributed decryption to be secure, in the sense that it reveals nothing more than the plaintext. We express this by requiring that there be a probabilistic polynomial time simulator $\mathcal{S}$. On input the public key ek and a corrupted (unqualified) set $\mathcal{I}$, $\mathcal{S}$ outputs a set of shares $\{s_i \mid i \in \mathcal{I}\}$. Then, each time it is given a ciphertext c and message m , it will output a set of decryption shares for the honest parties in $\mathcal{H}$: $\{d_i \mid i \in \mathcal{H}\}$.

We can now define the following game: the adversary chooses a corrupt set $\mathcal{I}$ (such that $\mathcal{I}$ is unqualified). The challenger then runs either the *real* game or the *ideal* game. In the real game, she runs ThGen and gives $\text{ek}, \{s_i \mid i \in \mathcal{I}\}$ to the adversary. The adversary may then choose pairs c, m where it must be the case that $c = \text{Enc}_{\text{ek}}(m)$; the challenger returns $\{\text{PartDec}_{s_i}(c) \mid i \in \mathcal{H}\}$.

The *ideal* version of the game is the same, except the challenger runs $\mathcal{S}$ instead of ThGen and PartDec .

We then require that the ideal and real games are statistically indistinguishable.

Note that if in the ideal game, we give to $\mathcal{S}$ $m', c = \text{Enc}_{\text{ek}}(m)$, where $m' \neq m$, $\mathcal{S}$ must produce a computationally indistinguishable distribution, or else we could break CPA security, since $\mathcal{S}$ sees only the public key.

5.3.3 Instantiations Linearly (threshold) homomorphic encryption with the properties we require here can be built, based on either the Paillier/Damgård-Jurik cryptosystem or class groups, details on this can be found in [Dam+25] for Paillier and in [BDO23] for class groups.

6 MPC Subprotocols

The subprotocols in this section are used in our MPC protocol. We assume that all parties hold a global public encryption key ek that is used for encrypting the inputs to the computation. A committee will run the subprotocols in order to decrypt one or more ciphertexts and pass on the secret decryption key dk to the next committee.

6.1 Decrypt

The Decrypt protocol can be run in parallel for as many ciphertexts as we like, so that the committee doing the decryption only sends one (set of) message(s).

Protocol Decrypt(c)

Assumptions: all parties hold the ciphertext $c = \text{Enc}_{\text{ek}}(x)$ to be decrypted, and the public keys $\text{ek}_1, \dots, \text{ek}_n$ of the committee assigned to do the decryption. Each committee member P_u holds dk_u . The committee holds $\text{VSS}(\text{dk}, \mathbf{v}_{\text{dk}}, r_1, \dots, r_\ell) = (s_1, c_1, \dots, s_\ell, c_\ell)$.

1. Each party P_u in the committee, for all i where $u = u(i)$, computes
 - $d_i = \text{PartDec}_{s_i}(c)$; and
 - a zero-knowledge proof

$$\pi_i = \text{NIZK}((\text{ek}_u, c, c_i, d_i); \text{dk}_u : d_i = \text{PartDec}_{s_i}(c) \text{ where } s_i = \text{Dec}_{\text{dk}_u}(c_i)).$$

P_i sends the decryption message $\{(d_i, \pi_i) | u = u(i)\}$ to all parties.

2. As soon as decryption messages with valid zero-knowledge proofs have been received from a qualified subset $\mathcal{H}$ of the committee, all parties compute and output

$$x = \text{PlainTxtRec}(\{d_i | u(i) \in \mathcal{H}\}).$$

To see why this works, note that, by soundness of the zero-knowledge proofs, accepted decryption messages from corrupt parties will be valid with overwhelming probability (note that PartDec is deterministic). Moreover, a qualified subset

of the committee can be assumed to be honest, and they will send correct decryption messages. Hence every party will receive enough decryption messages to finish the protocol correctly. Moreover, given the correct output plaintext, the protocol can be simulated, by first running the simulator $\mathcal{S}$ for threshold decryption to get decryption shares from honest parties, and then simulating their zero-knowledge proofs.

6.2 Reshare

The Reshare protocol is executed by three committees, C_{send} , C_{rec} and C_{aux} . C_{send} holds shares of the secret key dk , and the goal is for C_{rec} to obtain shares of dk . The idea, taken from [Dam+25], is that C_{aux} will create two sets of shares of the same random mask R and give shares to both C_{send} and C_{rec} . C_{send} then broadcasts shares of $R + \text{dk}$, so it can be reconstructed by all parties. Finally, C_{rec} can adjust their shares of R to get shares of dk . The point of this approach is that we can arrange it such that the size of shares output by C_{rec} only depends of the size of dk and not the number of times we reshare, as in earlier work.

Protocol Reshare

Assumptions: C_{send} holds $\text{VSS}(\text{dk}, \mathbf{v}_{\text{dk}})$, where $\text{dk} \in [0..2^b]$, and the randomness entries in $\mathbf{v}_{\text{dk}}$ are in $[0..2^{b+3\kappa+(c+2)m}]$. All parties hold public keys of members in C_{send} , C_{rec} and C_{aux} .

1. Each member P_i of C_{aux} : choose $R_i \in [0..2^{b+\kappa}]$ at random. Compute $c_{R_i} = \text{Enc}_{\text{ek}_i}(R_i)$.
2. P_i runs $\text{NonIntVSS}(b + \kappa, \text{ek}_i, \text{dk}_i, c_{R_i}, \text{ek}_1^{\text{rec}}, \dots, \text{ek}_n^{\text{rec}}) = (c_1^i, \dots, c_n^i, \pi_i^{\text{rec}})$.
3. P_i runs $\text{NonIntVSS}(b + 2\kappa + (c + 2)\log_2(m), \text{ek}_i, \text{dk}_i, c_{R_i}, \text{ek}_1^{\text{send}}, \dots, \text{ek}_n^{\text{send}}) = (d_1^i, \dots, d_n^i, \pi_i^{\text{send}})$.
4. P_i broadcasts both VSS messages. Let V be the set of P_i 's where both π_i^{rec} and π_i^{send} verify. Note that now C_{send} holds $\text{VSS}(R_i, \mathbf{v}_{R_i}^{\text{send}})$, and C_{rec} holds $\text{VSS}(R_i, \mathbf{v}_{R_i}^{\text{rec}})$ for all $P_i \in V$, where $\mathbf{v}_{R_i}^{\text{send}}$ and $\mathbf{v}_{R_i}^{\text{rec}}$ are the sharing vectors chosen by P_i .
5. Using local computation, C_{rec} computes

$$\text{VSS}(R, \mathbf{v}_R^{\text{rec}}) = \sum_{P_i \in V} \text{VSS}(R_i, \mathbf{v}_{R_i}^{\text{rec}}),$$

where we set R to be the sum of the R_i 's.

6. C_{send} computes

$$\text{VSS}(\text{dk} + R, \mathbf{v}_{\text{dk}+R}^{\text{send}}) = \text{VSS}(\text{dk}, \mathbf{v}_{\text{dk}}) + \sum_{P_i \in V} \text{VSS}(R_i, \mathbf{v}_{R_i}^{\text{send}}).$$

7. For each member P_j of C_{send} , let s_j be her share in $\text{VSS}(\text{dk} + R, \mathbf{v}_R^{\text{rec}})$ and $c_j = \text{Enc}_{\text{ek}_j^{\text{send}}}(s_j)$ be the ciphertext from the VSS containing s_j .

P_j computes a zero-knowledge proof:

$$\pi_j = \text{NIZK}(s_j, \text{ek}_j^{\text{send}}, \text{dk}_j^{\text{send}} : \text{Dec}_{\text{dk}_j^{\text{send}}}(c_j) = s_j)$$

and sends (s_j, π_j) to all parties.

8. All parties: once valid messages (s_j, π_j) have been received from a qualified subset of C_{send} , use the shares to reconstruct $R + \text{dk}$.
9. Using default (all-0) randomness $\mathbf{v}_{R+\text{dk}}$, form $\text{VSS}(R + \text{dk}, \mathbf{v}_{R+\text{dk}})$. Parties in C_{rec} then locally compute

$$\text{VSS}(\text{dk}, \mathbf{v}_{R+\text{dk}} - \mathbf{v}_R^{\text{rec}}) = \text{VSS}(R + \text{dk}, \mathbf{v}_{R+\text{dk}}) - \text{VSS}(R_i, \mathbf{v}_R^{\text{rec}})$$

We assume that the modulus M_{ind} used for the parties' individual encryption keys is chosen large enough such that all shares can be encrypted without causing overflow modulo M_{ind} . With this assumption, the correctness of the above protocol follows directly from the soundness of the zero-knowledge proofs and the fact that all committees can be assumed to contain a qualified set of honest parties.

For privacy, note that the adversary only learns an unqualified set of shares of any unopened VSS (as well as encryptions of the shared values, which we assume are secure). However, we do release a full set of shares of $R + \text{dk}$. We can assume that at least one honest party P_i chooses her R_i as a random $b + \kappa$ -bit number, and hence $R + \text{dk}$ is statistically close to a random $b + \kappa$ -bit number. As for the shares released, note that in step 3, P_i does the sharing with a larger bound on the secret size $(b + 2\kappa + (c + 2) \log m)$, and this allows the entries in $\mathbf{v}_{R_i}^{\text{send}}$ to be up to $b + 3\kappa + (c + 2) \log m$ bits. They are therefore κ bits larger than those in the original $\mathbf{v}_{\text{dk}}$ held by the sending committee. This “noise-drowns” the entries from the original $\mathbf{v}_{\text{dk}}$, more precisely, the sharing vector $\mathbf{v}_{\text{dk}+R}^{\text{send}}$ is statistically close to a fresh random sharing of $R + \text{dk}$. All this means that one can simulate the adversary's view of **Reshare** using a default value instead of dk .

Finally, the invariant is maintained, namely the receiving committee hold shares coming from a sharing vector with $b + 3\kappa + (c + 2) \log m$ bits. This is because the individual sharing vectors $\mathbf{v}_{R_i}^{\text{rec}}$ have entries with $(b + \kappa) + 2\kappa + c \log m$ bits, and then we form $\mathbf{v}_R^{\text{rec}}$ by adding at most $n = m^2$ of these.

6.3 Secure Multiplication

For secure multiplication, we need a subprotocol which starts from public ciphertexts $c_a = \text{Enc}_{\text{ek}}(a)$, $c_b = \text{Enc}_{\text{ek}}(b)$ and produces a new ciphertext $c_{ab} = \text{Enc}_{\text{ek}}(ab \bmod M)$ securely, that is, without leaking any side information. This is a standard protocol from the CDN paradigm, of which a YOSO instantiation can be found in [BDO23], for instance. It requires a constant number of committees to work in sequence.

For completeness, we sketch it here: Each member P_i of the first committee broadcasts a random ciphertext $c_i = \text{Enc}_{\text{ek}}(x_i)$ and a zero-knowledge proof of

plaintext knowledge. We set

$$c_x = \prod_i c_i = \text{Enc}_{\text{ek}}(\sum_i x_i) = \text{Enc}_{\text{ek}}(x),$$

where the product is over the parties whose proofs verify. Each party P_j in the next committee broadcasts a random ciphertext $c_j = \text{Enc}_{\text{ek}}(y_j)$ and a proof of plaintext knowledge. P_j also uses the homomorphic property to compute $d_j = \text{Enc}_{\text{ek}}(d_j x \bmod M)$ and does a proof that d_j was correctly formed. We then set

$$c_y = \prod_i c_j = \text{Enc}_{\text{ek}}(\sum_j x_j) = \text{Enc}_{\text{ek}}(y),$$

and

$$c_{xy} = \prod_i d_j = \text{Enc}_{\text{ek}}(\sum_j y_j x) = \text{Enc}_{\text{ek}}(xy),$$

again summing over the parties whose proofs verify. This forms a Beaver style multiplication triple c_x, c_y, c_{xy} , that the third and final committee can use in the standard way, namely given c_a, c_b one can compute $\text{Enc}_{\text{ek}}(a-x \bmod M)$, $\text{Enc}_{\text{ek}}(b-y \bmod M)$, and the committee runs the **Decrypt** protocol to reveal $a-x \bmod M$ and $b-y \bmod M$. Now all parties can use the homomorphic properties to compute $\text{Enc}_{\text{ek}}(ab \bmod M)$.

We use

$$c_{ab} = \text{Multiply}(c_a, c_b)$$

as shorthand for a call to this subprotocol.

7 CDN MPC

7.1 Functionalities

We will assume that the set of parties is split into committees $C_0, C_1, \dots$. We also assume a distinct set of special input parties $I_1, \dots, I_v$ whose only role is to supply inputs to the computation.

To get off the ground, we need a set-up functionality that gives key material to all parties and ensures that the global decryption key is shared among the members of the first committee.

If the homomorphic encryption is instantiated using class groups, one can implement the setup from scratch [BDO23], while using Paillier encryption requires set-up with current state of the art.

Functionality Setup

1. Run $\text{Gen}(1^\lambda)$ to get a key pair (ek, dk) , with modulus M and hand ek to all parties.

2. For all parties P_i , run $\text{Gen}(1^\lambda)$ to get a key pair $(\text{ek}_i, \text{dk}_i)$, with modulus M_{ind} , hand (i, ek_i) to all parties and dk_i to P_i .
3. Secret share dk for the first committee C_0 using sharing vector $\mathbf{v}_{\text{dk}}$, hand the shares to the members of C_0 , encrypt each share s_i under the owner's key $\text{ek}_{u(i)}$ and hand the ciphertexts to all parties. As a result C_0 now holds $\text{VSS}(\text{dk}, \mathbf{v}_{\text{dk}})$.

7.2 Protocols

The MPC protocol is executed by a set of committees working in sequence. We describe first how this works in the Committee Administration protocol, this allows us to later give a simpler specification of the actual MPC where we do not explicitly deal with the book-keeping on which committees do what.

Protocol Committee Administration

Initialize Set $i_{\text{active}} = 0$. This is the index of the currently active committee.

Auxiliary Committee acts. $C_{i_{\text{active}}+1}$ assumes the role of C_{aux} in the Reshare protocol, so each member broadcasts VSS messages intended for $C_{i_{\text{active}}}$ and $C_{i_{\text{active}}+2}$, who will later play the role of C_{send} and C_{rec} , respectively.

Current Committee acts. $C_{i_{\text{active}}}$ may be assigned to broadcast some ciphertexts, or decrypt some ciphertext. Each member P_i assembles a message contr_i which is its contribution to this task. In addition, P_i executes the Reshare protocol, acting as a member of C_{send} and accordingly assembles a message reshare_i . Finally, P_i broadcasts $\text{contr}_i, \text{reshare}_i$.

Future Committee Acts $C_{i_{\text{active}}+2}$ executes the final part of the Reshare protocol acting as C_{rec} . As a result, $C_{i_{\text{active}}+2}$ now holds a VSS of dk .

Advance index Set $i_{\text{active}} = i_{\text{active}} + 2$ and go to the “Auxiliary Committee acts” step.

In the MPC protocol below, We assume the function f to compute is given as an arithmetic circuit C_f over Z_M . For simplicity, we only consider the case of public outputs. It should be understood that while the MPC protocol runs, the Committee Administration protocol runs in the background, making sure that for each secure multiplication and/or decryption, a committee is ready to execute the required operation.

Protocol MPC

1. Invoke the **Setup** functionality.
2. Each input party I_j broadcasts $c_j = \text{Enc}_{\text{ek}}(x_j; r_j), \pi_j$, where $\pi_j = \text{NIZK}(\text{ek}, c_j; x_j, r_j : c_j = \text{Enc}_{\text{ek}}(x_j; r_j))$.
3. Those c_j for which π_j does not verify are replaced by encryptions of a default value using default randomness.
4. The parties work through the circuit C_f as follows:
 - (a) for all gates where encryptions c_a, c_b of both input values are available, compute an encryption of the output value as follows: for an addition gate, set $c_{a+b} = c_a + c_b$. For a multiplication gate, set $c_{ab} = \text{Multiply}(c_a, c_b)$.
 - (b) If not all gates have been processed, go to the previous step.
5. At this point, for each output y , a ciphertext $c_y = \text{Enc}_{\text{ek}}(y)$ is known to all parties. For all outputs, execute **Decrypt**(c_y) and output the result.

The proof of security of this protocol is standard and follows along the same lines as the proof of the corresponding protocols in [BDO23] or [Gen+21a]. We give a brief sketch here: the simulator will run a copy of the **Setup** functionality and will therefore know the global and all individual secret keys. Using this, it can extract inputs from corrupt parties by decrypting and define default inputs for honest parties. It then simulates straight-line by running the protocol until the output decryption step. At this point, the output ciphertexts likely contain incorrect values, but the simulator fabricates decryption shares from the honest parties that result in reconstructing the correct output.

We now want to claim that this inconsistency cannot be detected because the cryptosystem is CPA secure. To show this, one observes that the entire process can be emulated with statistically close distribution without knowing the secret key. Skipping some details, this can be done by using a default value in the VSS's instead of dk , and extracting inputs from the zero-knowledge proofs instead. The adversary only learns an unqualified set of shares in any VSS, and so cannot tell the difference.

We can now appeal to CPA security and switch the content of unopened ciphertext to whatever appears in the real protocol. In the final hybrid, we go back to using the real secret key dk again, and now we have arrived at the view of the real protocol.

References

- [Abr+25] Ittai Abraham, Eli Chouatt, Yossi Gilad, Gilad Stern, and Sophia Yakoubov. “Asynchronous Algorand: Reaching Agreement with Near Linear Communication and Constant Expected Time”. In: *44th ACM Symposium Annual on Principles of Distributed Computing*. Ed. by Alkida Balliu and Fabian Kuhn. Huatulco, Mexico: As-

- sociation for Computing Machinery, June 2025, pp. 28–38. DOI: 10.1145/3732772.3733533.
- [BDO23] Lennart Braun, Ivan Damgård, and Claudio Orlandi. “Secure Multiparty Computation from Threshold Encryption Based on Class Groups”. In: *Advances in Cryptology – CRYPTO 2023, Part I*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14081. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2023, pp. 613–645. DOI: 10.1007/978-3-031-38557-5_20.
- [Ben+20] Fabrice Benhamouda, Craig Gentry, Sergey Gorbunov, Shai Halevi, Hugo Krawczyk, Chengyu Lin, Tal Rabin, and Leonid Reyzin. “Can a Public Blockchain Keep a Secret?” In: *TCC 2020: 18th Theory of Cryptography Conference, Part I*. Ed. by Rafael Pass and Krzysztof Pietrzak. Vol. 12550. Lecture Notes in Computer Science. Durham, NC, USA: Springer, Cham, Switzerland, Nov. 2020, pp. 260–290. DOI: 10.1007/978-3-030-64375-1_10.
- [Can+23] Ran Canetti, Sebastian Kolby, Divya Ravi, Eduardo Soria-Vazquez, and Sophia Yakubov. “Taming Adaptivity in YOSO Protocols: The Modular Way”. In: *TCC 2023: 21st Theory of Cryptography Conference, Part II*. Ed. by Guy N. Rothblum and Hoeteck Wee. Vol. 14370. Lecture Notes in Computer Science. Taipei, Taiwan: Springer, Cham, Switzerland, Nov. 2023, pp. 33–62. DOI: 10.1007/978-3-031-48618-0_2.
- [Can01] Ran Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols”. In: *42nd Annual Symposium on Foundations of Computer Science*. Las Vegas, NV, USA: IEEE Computer Society Press, Oct. 2001, pp. 136–145. DOI: 10.1109/SFCS.2001.959888.
- [Cas+22] Ignacio Cascudo, Bernardo David, Lydia Garms, and Anders Konring. “YOLO YOSO: Fast and Simple Encryption and Secret Sharing in the YOSO Model”. In: *Advances in Cryptology – ASIACRYPT 2022, Part I*. Ed. by Shweta Agrawal and Dongdai Lin. Vol. 13791. Lecture Notes in Computer Science. Taipei, Taiwan: Springer, Cham, Switzerland, Dec. 2022, pp. 651–680. DOI: 10.1007/978-3-031-22963-3_22.
- [CDN01] Ronald Cramer, Ivan Damgård, and Jesper Buus Nielsen. “Multiparty Computation from Threshold Homomorphic Encryption”. In: *Advances in Cryptology – EUROCRYPT 2001*. Ed. by Birgit Pfitzmann. Vol. 2045. Lecture Notes in Computer Science. Innsbruck, Austria: Springer Berlin Heidelberg, Germany, May 2001, pp. 280–299. DOI: 10.1007/3-540-44987-6_18.
- [CKS05] Christian Cachin, Klaus Kursawe, and Victor Shoup. “Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement Using Cryptography”. In: *Journal of Cryptology* 18.3 (July 2005), pp. 219–246. DOI: 10.1007/s00145-005-0318-0.

- [Cle86] Richard Cleve. “Limits on the Security of Coin Flips when Half the Processors Are Faulty (Extended Abstract)”. In: *18th Annual ACM Symposium on Theory of Computing*. Berkeley, CA, USA: ACM Press, May 1986, pp. 364–369. DOI: 10.1145/12130.12168.
- [Dam+25] Ivan Bjerre Damgård, Simon Holmgård Kamp, Julian Loss, and Jesper Buus Nielsen. *Asynchronous YOSO a la Paillier*. Cryptology ePrint Archive, Report 2025/128. 2025. URL: <https://eprint.iacr.org/2025/128>.
- [Del+24] Giovanni Deligios, Anders Konring, Chen-Da Liu-Zhang, and Varun Narayanan. “Statistical Layered MPC”. In: *TCC 2024: 22nd Theory of Cryptography Conference, Part IV*. Ed. by Elette Boyle and Mohammad Mahmoody. Vol. 15367. Lecture Notes in Computer Science. Milan, Italy: Springer, Cham, Switzerland, Dec. 2024, pp. 362–394. DOI: 10.1007/978-3-031-78023-3_12.
- [Det+25] Gabriel Dettling, Chen-Da Liu-Zhang, Elisaweta Masserova, Matthieu Rambaud, and Antoine Urban. *Broadcast for Dynamic Committees without Trusted Setup*. Cryptology ePrint Archive, Paper 2025/2078. 2025. URL: <https://eprint.iacr.org/2025/2078>.
- [DLS84] Cynthia Dwork, Nancy A. Lynch, and Larry J. Stockmeyer. “Consensus in the Presence of Partial Synchrony (Preliminary Version)”. In: *3rd ACM Symposium Annual on Principles of Distributed Computing*. Ed. by Robert L. Probert, Nancy A. Lynch, and Nicola Santoro. Vancouver, BC, Canada: Association for Computing Machinery, Aug. 1984, pp. 103–118. DOI: 10.1145/800222.1599406.
- [DT06] Ivan Damgård and Rune Thorbek. “Linear Integer Secret Sharing and Distributed Exponentiation”. In: *PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography*. Ed. by Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin. Vol. 3958. Lecture Notes in Computer Science. New York, NY, USA: Springer Berlin Heidelberg, Germany, Apr. 2006, pp. 75–90. DOI: 10.1007/11745853_6.
- [Gen+21a] Craig Gentry, Shai Halevi, Hugo Krawczyk, Bernardo Magri, Jesper Buus Nielsen, Tal Rabin, and Sophia Yakoubov. “YOSO: You Only Speak Once - Secure MPC with Stateless Ephemeral Roles”. In: *Advances in Cryptology – CRYPTO 2021, Part II*. Ed. by Tal Malkin and Chris Peikert. Vol. 12826. Lecture Notes in Computer Science. Virtual Event: Springer, Cham, Switzerland, Aug. 2021, pp. 64–93. DOI: 10.1007/978-3-030-84245-1_3.
- [Gen+21b] Craig Gentry, Shai Halevi, Bernardo Magri, Jesper Buus Nielsen, and Sophia Yakoubov. “Random-Index PIR and Applications”. In: *TCC 2021: 19th Theory of Cryptography Conference, Part III*. Ed. by Kobbi Nissim and Brent Waters. Vol. 13044. Lecture Notes in Computer Science. Raleigh, NC, USA: Springer, Cham, Switzerland, Nov. 2021, pp. 32–61. DOI: 10.1007/978-3-030-90456-2_2.

- [Gil+17] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. *Algorand: Scaling Byzantine Agreements for Cryptocurrencies*. Cryptology ePrint Archive, Report 2017/454. 2017. URL: <https://eprint.iacr.org/2017/454>.
- [Hua+22] Zhengan Huang, Junzuo Lai, Shuai Han, Lin Lyu, and Jian Weng. “Anonymous Public Key Encryption Under Corruptions”. In: *Advances in Cryptology – ASIACRYPT 2022, Part III*. Ed. by Shweta Agrawal and Dongdai Lin. Vol. 13793. Lecture Notes in Computer Science. Taipei, Taiwan: Springer, Cham, Switzerland, Dec. 2022, pp. 423–453. DOI: 10.1007/978-3-031-22969-5_15.
- [Kat+13] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. “Universally Composable Synchronous Computation”. In: *TCC 2013: 10th Theory of Cryptography Conference*. Ed. by Amit Sahai. Vol. 7785. Lecture Notes in Computer Science. Tokyo, Japan: Springer Berlin Heidelberg, Germany, Mar. 2013, pp. 477–498. DOI: 10.1007/978-3-642-36594-2_27.
- [MRV99] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. “Verifiable Random Functions”. In: *40th Annual Symposium on Foundations of Computer Science*. New York, NY, USA: IEEE Computer Society Press, Oct. 1999, pp. 120–130. DOI: 10.1109/SFFCS.1999.814584.
- [Ram23] Matthieu Rambaud. *Adaptively Secure Consensus with Linear Complexity and Constant Round under Honest Majority in the Bare PKI Model, and Separation Bounds from the Idealized Message-Authentication Model*. Cryptology ePrint Archive, Report 2023/1757. 2023. URL: <https://eprint.iacr.org/2023/1757>.
- [RSY21] Leonid Reyzin, Adam Smith, and Sophia Yakubov. “Turning HATE into LOVE: Compact Homomorphic Ad Hoc Threshold Encryption for Scalable MPC”. In: *Cyber Security Cryptography and Machine Learning*. Ed. by Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander Schwarzmann. Cham: Springer International Publishing, 2021, pp. 361–378. ISBN: 978-3-030-78086-9.
- [Sah99] Amit Sahai. “Non-Malleable Non-Interactive Zero Knowledge and Adaptive Chosen-Ciphertext Security”. In: *40th Annual Symposium on Foundations of Computer Science*. New York, NY, USA: IEEE Computer Society Press, Oct. 1999, pp. 543–553. DOI: 10.1109/SFFCS.1999.814628.
- [Sha79] Adi Shamir. “How to Share a Secret”. In: *Communications of the Association for Computing Machinery* 22.11 (Nov. 1979), pp. 612–613. DOI: 10.1145/359168.359176.
- [Sta96] Markus Stadler. “Publicly Verifiable Secret Sharing”. In: *Advances in Cryptology – EUROCRYPT’96*. Ed. by Ueli M. Maurer. Vol. 1070. Lecture Notes in Computer Science. Saragossa, Spain: Springer Berlin Heidelberg, Germany, May 1996, pp. 190–199. DOI: 10.1007/3-540-68339-9_17.

A Definitions for Secure Multiparty Computation

A protocol Π can be thought of as a tuple of probabilistic polynomial-time (PPT) participants $(P_1, \dots, P_u)$. In this section we recap what it means for a protocol to be (YOSO) secure [Gen+21a]. In the YOSO model, every participant is “YOSO-ified”, meaning that it is shut down as soon as it speaks for the first time. A protocol YOSO-realizes a functionality $\mathcal{F}$ if the YOSO-ification of that protocol UC-realizes $\mathcal{F}$.

Communication Model We assume a synchronous communication model in which the protocol proceeds in rounds, with certain participants speaking (via a broadcast channel) during each round. The adversary $\mathcal{A}$ is assumed to be rushing, meaning it can observe the honest participants’ messages in a round before generating its own for the corrupted participants.

Real-World Execution In a real-world execution of protocol Π with environment $\mathcal{Z}$ and adversary $\mathcal{A}$, $\mathcal{Z}$ supplies the inputs, and interacts with $\mathcal{A}$. Throughout the execution, corrupt participants follow arbitrary instructions from $\mathcal{A}$, while honest participants adhere strictly to the protocol using the inputs provided by $\mathcal{Z}$. At the end of the protocol, $\mathcal{Z}$ outputs a single bit. We let $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}}(1^\lambda)$ denote the random variable that represents $\mathcal{Z}$ ’s output in this real-world execution.

Ideal-World Execution In an ideal-world execution, the same environment $\mathcal{Z}$ runs with an ideal adversary $\mathcal{S}$ (the simulator). Instead of executing protocol Π , the participants invoke the trusted ideal functionality $\mathcal{F}$ on the inputs provided by $\mathcal{Z}$. This functionality collects the inputs, and distributes the outputs. Let $\text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ denote the random variable that represents $\mathcal{Z}$ ’s output in this ideal-world execution.

Definition 1 (UC Security [Can01]). *Let $f : (\{0, 1\}^*)^l \rightarrow \{0, 1\}^*$ be a l -input function. A protocol $\Pi = (P_1, \dots, P_u)$ UC-securely computes f (with guaranteed output delivery) if for every PPT real-world adversary $\mathcal{A}$ there exists a PPT ideal-world adversary (or simulator) $\mathcal{S}$ such that, for any PPT environment $\mathcal{Z}$, it holds that $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}}(1^\lambda)$ and $\text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ are indistinguishable for any large enough security parameter λ .*

In this paper, we present two protocols: a *role assignment* protocol (Section 3) (presented in terms of participants who are real-world machines) that YOSO-realizes the ideal functionality $\mathcal{F}_{\text{RA-general}}$ (Section 2) with security against *adaptive* adversaries, and a *general MPC* protocol (Section 7) (presented in terms of participants who are abstract-world roles) that YOSO-realizes the ideal functionality $\mathcal{F}_f$ (for any efficient function f) with security against *static* adversaries. $\mathcal{F}_f$ collects inputs, computes f on those inputs to obtain an output y , and distributes y . $\mathcal{F}_f$ has *guaranteed output delivery*, meaning that no ideal adversary $\mathcal{S}$ is able to prevent honest participants from getting y .

B Building Blocks for Role Assignment

In this section, we briefly recall the building blocks we leverage to build a general role assignment functionality: namely, a random beacon (Appendix B.1), a verifiable random function (VRF, Appendix B.2, [MRV99]), and a key-message non-committing encryption scheme (KMNCE, Appendix B.3, [Hua+22]). We refer to the papers in which these primitives are introduced for formal definitions.

VRF and KMNCE schemes are both equipped with a setup algorithm **Setup** that generates public parameters $\mathbf{pp}$ for the scheme; these public parameters are then given to all other algorithms. For notational simplicity, we omit both **Setup** and $\mathbf{pp}$, leaving them implicit.

B.1 Random Beacon

In our role assignment protocol, we need two different random beacons: a *programmable* random beacon to permute the roles, and an *unpredictable* random beacon (which we refer to simply as “random beacon”), within sortition. The programmable random beacon (B.1.1) is strictly stronger than the random beacon (B.1.2), and can be realized using sortition.

This might seem like a chicken-and-egg problem, since we use sortition to build our programmable random beacon, but a random beacon is in turn necessary to realize sortition. However, this is not the case, since we use sortition to build the *programmable* random beacon, and we only need an *unpredictable* random beacon to realize sortition.

We use one instance of the weaker (unpredictable) random beacon, at the very beginning, to realize our first sortition (the stronger programmable random beacon is available for future instances, bootstrapping previous sortitions).

B.1.1 Programmable Random Beacon We model the programmable random beacon as an ideal functionality $\mathcal{F}_{\text{beacon}}$.

Functionality $\mathcal{F}_{\text{beacon}}$

This functionality produces randomness at every round. It maintains a mapping **randomness** from round number to the randomness produced in that round. At initialization, it creates an empty mapping **randomness**.

Read: On input (Read, r') from party P in round r :

- If $r' > r$: ignore this query.
- If no randomness is stored for round r' : sample fresh randomness ρ , and record $\text{randomness}[r'] = \rho$.
- Return $\text{randomness}[r']$ to P .

Instantiation To instantiate our programmable random beacon, we need several tools:

1. A cryptographic sortition (Appendix B.2.3). (This, in turn, can be realized using an unpredictable beacon together with a VRF (Appendix B.2).)
2. A key-message non-committing encryption scheme (KMNCE, Appendix B.3), with a public KMNCE key for each participant available (PKI-style).
3. A secret sharing scheme that is
 - (a) one-round;
 - (b) nested (meaning that qualified shareholder subsets follow the nested access structure, described in Section 3.1); and
 - (c) share-simulatable [RSY21] (meaning that, given all public information, an unqualified subset of shares and a target secret, remaining shares can be simulated – given the necessary trapdoors — that appear consistent with the given information).

The scheme described in Section 5.2.2 satisfies all these properties.

4. A non-interactive zero-knowledge (NIZK) proof scheme (Section 5.1).
5. An IND-CPA public-key encryption (PKE) scheme.

Protocol Programmable Random Beacon (Informal)

Round 1 (Nomination):

1. Use sortition to get an honest majority committee, which we will call the *nominating committee*.
2. Each member of the nominating committee picks a random sub-committee (whose size is set to guarantee an honest majority if members are picked randomly). These committees together comprise the *holding committee*. The nominating committee member generates a fresh PKE key-pair for each member of its holding sub-committee; it publishes the public keys, and communicates the secret keys to the chosen sub-committee members using key-message non-committing encryption (Appendix B.3).

Round 2 (Commit):

1. Use sortition to get a dishonest majority committee, which we will call the *choosing committee*.
2. Each member of the choosing committee picks a random value and secret shares the random value, encrypting the shares to the keys published by the choosing committee, and including NIZK proofs of correct sharing and encryption. (This gives us a publicly verifiable secret sharing — PVSS [Sta96].)

Round 3 (Open): Each member of the holding committee decrypts and broadcasts the shares it receives, along with NIZK proofs of correct decryption.

Output Computation: Parties reconstruct all random values (whose NIZK proofs of correct sharing verify) using the broadcasted shares, and output their sum.

Theorem 2. *The programmable random beacon protocol realizes $\mathcal{F}_{\text{beacon}}$.*

Proof (sketch). The unpredictability of the sortition, together with the properties of the KMNCE scheme, can be used to argue that an adaptive adversary cannot exceed the corruption threshold of each individual committee. This is done just as in the proof of Theorem 1; in this proof sketch, we omit these hybrids, instead taking as a starting point a world where each committee is guaranteed to have the desired number of honest parties.

In such a world, the simulator can force the output of the programmable random beacon protocol to be a target value r , by running rounds 1 (nomination) and 2 (commit) honestly, extracting all random values chosen by the corrupt choosing committee members (the honest holding committee members' views suffice for this), and, in round 3 (open), equivocating the random value s chosen by the one honest member of the choosing committee such that the random values sum to r .

To do this, the simulator does the following on behalf of the holding committee:

- For the one honest choosing committee member's sharing:
 - Use share-simulatability to get honest holding committee members' simulated shares (from the corrupt holding committee members' shares, together with the target random value s).
 - Instruct the simulated honest holding committee members to broadcast the simulated shares along with simulated proofs of correct opening. To generate these proofs, the simulator uses its trapdoor to the non-interactive zero knowledge proof and key-message non-committing encryption schemes.
- For all other sharings, instruct the simulated honest holding committee members to honestly follow the protocol and broadcast the correct values chosen by the adversary.

We next sketch a sequence of hybrids arguing the indistinguishability of the real and ideal worlds.

Hybrid $\mathcal{H}_{\text{real}}$: We start with the real world, where the real programmable random beacon protocol is run between the adversary and the honest parties. Let s be the random value chosen by the one honest member of the choosing committee.

Hybrid $\mathcal{H}_{\text{nizksim}}$: This is the same as the previous hybrid, except the simulator simulates all honest choosing and holding committee members' NIZK proofs. This is indistinguishable from the previous hybrid, by the zero knowledge property of the NIZK proofs.

Hybrid $\mathcal{H}_{\text{indcpa}}$: This is the same as the previous hybrid, except that the simulator does not encrypt honest holding committee member shares of s ; rather, it encrypts $\perp$ to each honest holding committee member. The honest holding committee members still publish the correct shares (though they no longer get those shares from the honest choosing committee member's ciphertexts).

Since the NIZKs are already simulated, this is indistinguishable from the previous hybrid by the IND-CPA security of the encryption scheme.

Hybrid $\mathcal{H}_{\text{sharesim}}$: This is the same as the previous hybrid, except that the simulator discards the original honest holding committee members' shares of s ; when it comes time to execute round 3, the simulator uses share-simulatability to simulate honest holding committee members' shares of s . This is indistinguishable from the previous hybrid by the share-simulatability of the secret sharing scheme.

Hybrid $\mathcal{H}_{\text{random}}$: This is the same as the previous hybrid, except that the simulator uses share-simulatability to derive shares of a fresh randomly-chosen s' rather than the originally-shared s . This is indistinguishable from the previous hybrid by the hiding property of the secret sharing scheme, since the adversary only has access to an unqualified set of shares of s .

Hybrid $\mathcal{H}_{\text{ideal}}$: This is the same as the previous hybrid, except now the simulator invokes the ideal functionality $\mathcal{F}_{\text{beacon}}$ to get the random value ρ . It sets s' to be exactly ρ minus all corrupt choosing committee members' values. This is indistinguishable from the previous hybrid because s' is uniformly randomly distributed in both.

We have now arrived at the ideal world.

B.1.2 Random Beacon We need one random beacon value right off the bat: it should be revealed right after the VRF keys are published, and will be used to seed the very first cryptographic sortition.

We can instantiate this beacon by inputting all generated public VRF keys to a random oracle. Of course, this gives a rushing adversary the power to rejection-sample: it can generate corrupt parties' keys, test what the random oracle would be (on those keys together with the honest parties' keys), and if it doesn't like the output, it can try again. However, this kind of rejection-sampleable beacon is sufficient for use in conjunction with a VRF to instantiate a cryptographic sortition (Appendix B.2.3).

B.2 Verifiable Random Function (VRF)

Verifiable random functions (VRFs) were introduced by Micali, Rabin and Vadhan [MRV99]. A VRF allows the owner of a key pair (vk, sk) to produce unique randomness associated with an input x that looks uniform given only vk , but which is verifiable when additionally given a proof π .

B.2.1 Syntax A VRF consists of three algorithms:

Gen $(1^\lambda) \rightarrow (\text{vk}, \text{sk})$: Given the security parameter λ , the key generation algorithm produces a public verification key vk and a secret evaluation key sk .
Eval $(\text{sk}, x) \rightarrow (\text{draw}, \pi)$: Given a secret evaluation key sk and an input x , the evaluation algorithm produces a draw draw , and a proof π .

$\text{Ver}(\text{vk}, x, \text{draw}, \pi) \rightarrow \{\text{accept}, \text{reject}\}$: Given a public verification key vk , an input x , a draw draw and a proof π , the verification algorithm determines whether draw was a valid evaluation of the VRF on input x with the secret key associated with vk .

B.2.2 Properties A VRF should satisfy three properties: correctness, soundness, and pseudorandomness.

Correctness: Ver should always accept an honest output of Eval .

Soundness / Uniqueness: It should be infeasible to create a public key vk^* along with $x, \text{draw}_0, \pi_0, \text{draw}_1, \pi_1$ s.t. $\text{draw}_0 \neq \text{draw}_1$ and $\text{Ver}(\text{vk}^*, x, \text{draw}_0, \pi_0) = \text{Ver}(\text{vk}^*, x, \text{draw}_1, \pi_1) = \text{accept}$.

Pseudorandomness: Without π , the output draw of Eval should be indistinguishable from random.

B.2.3 Cryptographic Sortition A VRF is commonly used to instantiate a *cryptographic sortition* [Gil+17], where all participants evaluate their individual VRFs on a common input x (often taken to be the output of a random beacon), and the winners are those whose participants whose draws are “small enough”. If the desired winning probability is p , then a winning draw must be smaller than $p2^{\ell_{\text{VRF}}}$, where ℓ_{VRF} is the size of the VRF output.

If the input to the VRF becomes known after the keys are generated, we have this guarantee even if some keys were generated maliciously [MRV99; Ben+20].

B.3 Key-Message Non-Committing Encryption (KMNCE)

Key-Message Non-Committing Encryption (KMNCE, [Hua+22]) is a kind of public-key encryption scheme which allows the generation of “fake” (but real-looking) ciphertexts that can be explained as an encryption of *any* message to *any* public encryption key, by generating a corresponding “fake” (but real-looking) decryption key.

B.3.1 Syntax A KMNCE scheme consists of five algorithms:

$\text{Gen}(1^\lambda) \rightarrow (\text{ek}, \text{dk}, \text{tk})$: Given the security parameter λ , the key generation algorithm produces a public encryption key ek and a secret decryption key dk , as well as a trapdoor key tk . (The trapdoor key is only used for opening simulated ciphertexts — not for real encryption or decryption.)

$\text{Enc}(\text{ek}, m) \rightarrow c$: Given a public encryption key ek and a message m , the encryption algorithm produces a ciphertext c .

$\text{Dec}(\text{dk}, c) \rightarrow m$: Given a secret decryption key dk and a ciphertext c , the decryption algorithm outputs a message m .

$\text{Fake}() \rightarrow (c, \text{td})$: Given only the public parameters pp , the fake algorithm produces a fake ciphertext c and additional trapdoor information td .

Open(tk, ek, dk, $(c_\gamma^*, \text{td}_\gamma^*, m_\gamma^*)_{\gamma \in [k]}$) $\rightarrow \text{dk}'$: Given keys tk, ek, dk, and k tuples, each containing a ciphertext c_γ , its trapdoor information td_γ and a desired plaintext m_γ , the open algorithm produces a fresh secret key dk' corresponding to ek, such that each ciphertext appropriately decrypts to the desired plaintext.

B.3.2 Properties There are several desirable properties for a KMNCE scheme: correctness, k -CCA security, and recipient uniqueness.

Correctness: The decryption of an encryption of m should return m .

k -CCA Security [Hua+22]: It should be infeasible for an adversary (with access to a decryption oracle, and with the freedom to choose up to k target messages) to distinguish between the outputs of **Enc** (and the real decryption key) and the outputs of **Fake** (together with the output of **Open**).

This property implies traditional CCA security.

Recipient Uniqueness [Can+23]: It should be infeasible to come up with a ciphertext that decrypts (i.e., does not output $\perp$) under two different honestly-generated secret decryption keys.

B.3.3 Instantiation We use the KMNCE construction of Canetti *et. al* [Can+23], which uses a programmable random oracle to achieve k -CCS Security for any k polynomial in the security parameter λ .

C Proof of Theorem 1

We proceed in a sequence of hybrids largely analogous to those in Canetti *et. al* [Can+23].

Hybrid $\mathcal{H}_{\text{real}}$: The real protocol $\Pi_{\text{RA-nested}}(\mathcal{P}, n = m^2, \mathcal{D}, \text{delay})$ is run; the simulator executes the protocol on behalf of honest parties.

Hybrid $\mathcal{H}_{\text{goodnom}}$: Same as before, except for each committee cid, the simulator keeps a counter b_{cid} of how many corrupt parties win the sortition (which determines the right to nominate parties to the committee). It aborts if $b_{\text{cid}} > tm$.

This hybrid is indistinguishable from the previous one because an adversary who is able to cause an abort is able to break the cryptographic sortition.

Hybrid $\mathcal{H}_{\text{uniquedec}}$: Same as before, except the simulator aborts if any ciphertext is decryptable by more than one honest party.

This hybrid is indistinguishable from the previous one as a direct consequence of the unique recipient property of KMNCE.

Hybrid $\mathcal{H}_{\text{fakeenc}}$: For each KMNCE encryption from an honest sender to a currently honest receiver, the simulator produces the ciphertext using the faking algorithm. (This is actually several hybrids, described in one; we introduce one hybrid for each honest KMNCE receiver.)

If the receiver is corrupted before it is given the next **Delete** command, the simulator (which generates — and thus knows — all honest parties'

secret keys) may use the open algorithm to ensure that the faked ciphertexts decrypt correctly.

Note that the simulator no longer touches an honest role's private key before the protocol completes, unless that honest role gets corrupted.

This hybrid is indistinguishable from the previous one by a reduction to the KMNCE-CCA security of the encryption scheme.

The reduction gets the public key for the receiver from the KMNCE-CCA challenger; tells the challenger which messages to encrypt; posts the received ciphertexts; and, upon corruption, it gets the receiver's secret key from the challenger (which it then passes on to the distinguishing adversary). If the adversary guesses that it is in the previous hybrid, the reduction guesses that it is in the real game; if the adversary guesses that it is in this hybrid, the reduction guesses that it is in the ideal game. The reduction succeeds whenever the adversary does.

Hybrid $\mathcal{H}_{\text{lazy sample}}$: The simulator does not choose which honest parties have been nominated by honest nominators until nomination is complete. That is, when making a nomination on behalf of an honest party, with probability $\frac{|Z|}{|P|}$ the simulator follows the protocol nominating a random *corrupt* party; with probability $\frac{|H|}{|P|}$ the simulator fakes a ciphertext without choosing a recipient. When the adversary makes a new corruption before the completion of the protocol, the simulator assigns each honest nomination to the newly corrupted party with probability $\frac{1}{|H|}$.

This hybrid is indistinguishable from the previous one because the only thing that changes is when the simulator's internal choices are made — not their distribution.

Hybrid $\mathcal{H}_{\text{good com}}$: Same as before, except for each committee cid , the simulator keeps track of the corrupt nominations via a list ccor_{cid} of n bits. (An honest party nominated by a corrupt party is counted as a corrupt nomination.) The simulator enforces bounds on corrupt nominations: if $A(\text{ccor}_{\text{cid}}) = 0$ at any point, the simulator aborts.

We are already guaranteed to have an honest majority of nominators for each committee (see hybrid $\mathcal{H}_{\text{good nom}}$). To show that this hybrid is indistinguishable from the previous one, we need only argue that an honest nominator nominates a sub-committee with an honest majority with overwhelming probability. This follows directly from our choice of m .

Hybrid $\mathcal{H}_{\text{use func}}$: In this hybrid, the simulator starts using $\mathcal{F}$, as detailed in the simulator below. When nominating on behalf of an honest party, it sends **NomHonest** with probability $\frac{|H|}{|P|}$, and **NomCorrupt** with probability $\frac{|Z|}{|P|}$. Upon receiving a nomination from a corrupt party, it passes it on to $\mathcal{F}$ using **NomCorrupt**. Upon receiving a corruption, it corrupts nominees with the appropriate probability using **CorruptNominee**.

This hybrid is indistinguishable from the previous one because all the same logic is executed — except that some of it is moved to the functionality $\mathcal{F}$.

Simulator $\mathcal{S}_{\text{RA-nested}}$

We present a simulator for a real world adversary $\mathcal{A}_{\text{RA-nested}}$.

For each committee cid , the simulator keeps track of:

- b_{cid} (initially set to 0): How many corrupt nominations happened. (*From $\mathcal{H}_{\text{goodnom}}$.*)
- noms_{cid} (initially set to 0): How many subcommittees have already been nominated.
- ccor_{cid} (initially set to $[0, \dots, 0]$): Which roles on the committee are corrupt. (*From $\mathcal{H}_{\text{goodcom}}$.*)

The simulator knows the KMNCE trapdoor tk , all parties' KMNCE and VRF public keys, and honest parties' KMNCE and VRF secret keys.

The simulator interacts with the ideal functionality as follows:

New Committee:

1. Send (New, cid) to $\mathcal{F}$ on behalf of all honest parties.
2. When an honest party P_{pid} , given (New, cid) , wins the sortition (with draw, π):
 - (a) Increment noms_{cid} , and set $i = \text{noms}_{\text{cid}}$.
 - (b) For $j \in [m]$:
 - i. With probability $\frac{|\mathcal{H}|}{|\mathcal{P}|}$: (*Nominate an honest party without learning its identity or secret key.*)
 - A. Input $(\text{NomHonest}, \text{cid}, R = (i, j))$ to $\mathcal{F}$.
 - B. Receive $(\text{NomHonest}, R = \text{cid}, (i, j), \text{pk}_{i,j})$ from $\mathcal{F}$.
(*Note that $\mathcal{F}$ chooses $\text{pk}_{i,j}$ via Gen, just like an honest party would.*)
 - C. Fake an KMNCE ciphertext (*as per $\mathcal{H}_{\text{fakeenc}}$*):

$$(c_{i,j}, \text{td}_{i,j}) \leftarrow \text{KMNCE.Fake}().$$
 - ii. Otherwise: (*Nominate a corrupt party.*)
 - A. Set $\text{ccor}_{\text{cid}}[(i, j)] = 1$. If $A(\text{ccor}_{\text{cid}}) = 0$: abort (*as per $\mathcal{H}_{\text{goodcom}}$*).
 - B. Choose a uniformly random corrupt party $\text{pid}_{i,j} \leftarrow \mathcal{I}$.
 - C. Sample fresh keys:

$$(\text{pk}_{i,j}, \text{sk}_{i,j}) \leftarrow \text{Gen}().$$
 - D. Input $(\text{NomCorrupt}, \text{cid}, R = (i, j), \text{pid}_{i,j}, \text{pk}_{i,j}, \text{sk}_{i,j})$ to $\mathcal{F}$.
 - E. Encrypt the secret key to the chosen party:

$$c_{i,j} \leftarrow \text{KMNCE.Enc}(P_{\text{pid}_{i,j}}.\text{ek}, (\text{pid}_{i,j}, \text{sk}_{i,j})).$$

(c) Post

$$\left(\text{cid}, \text{pid}, \text{draw}, \pi, (\text{pk}_{i,1}, c_{i,1}), \dots, (\text{pk}_{i,m}, c_{i,m}) \right)$$

to the bulletin board.

3. When $\mathcal{A}_{\text{RA-nested}}$ posts to the bulletin board on behalf of corrupt party P_{pid^*} :
 - (a) If draw is not a winning draw: ignore the post.
Else: increment noms_{cid} , and set $i = \text{noms}_{\text{cid}}$.

- (b) Attempt to decrypt each posted ciphertext $c_{i,j}$ on behalf of each honest party P_{pid} .
 - i. If the ciphertext correctly decrypts for more than one honest party, abort (*as per $\mathcal{H}_{\text{uniquedec}}$*).
 - ii. Set $\text{ccor}_{\text{cid}}[(i, j)] = 1$. If $A(\text{ccor}_{\text{cid}}) = 0$: abort (*as per $\mathcal{H}_{\text{goodcom}}$*).
 - iii. If $(\text{pid}, \text{sk}) = \text{KMNCE.Dec}(P_{\text{pid}}.\text{sk}, c_{i,j})$, input $(\text{NomCorrupt}, \text{cid}, R = (i, j), \text{pid}, \text{pk}, \text{sk})$ to $\mathcal{F}$. (*P_{pid^*} nominated honest P_{pid} .*)
 - iv. If it only decrypts to $\perp$ for honest parties, input $(\text{NomCorrupt}, \text{cid}, R = (i, j), \perp, \text{pk}, \perp)$ to $\mathcal{F}$. (*This nomination is either malformed or corrupt. The simulator and functionality make no distinction between those two cases.*)

Read: On behalf of each honest party P_{pid} , input $(\text{Read}, \text{cid})$ to $\mathcal{F}$. Receive $(\text{pkeys}, \text{skeys}_{\text{pid}})$.

Corrupt: When $\mathcal{A}_{\text{RA-nested}}$ adaptively corrupts P_{pid^*} :

1. For each honest nomination of role $R = (i, j)$, assign R to pid^* with probability $\frac{1}{|\mathcal{H}|}$. When R is assigned to pid^* , input $(\text{CorruptNominee}, \text{cid}, R, \text{pid}^*)$ to $\mathcal{F}$. Receive $(\text{CorruptNominee}, \text{cid}, R, \text{pid}^*, \text{sk})$. Obtain a secret key for the party using the KMNCE open algorithm (*as per $\mathcal{H}_{\text{fakeenc}}$*):

$$P_{\text{pid}^*}.\text{sk}' \leftarrow \text{KMNCE.Open}(\text{tk}, P_{\text{pid}^*}.\text{pk}, P_{\text{pid}^*}.\text{sk}, (c, \text{td}, m = (\text{pid}^*, \text{sk}))).$$

2. If $A(\text{ccor}_{\text{cid}}) = 0$: abort (*as per $\mathcal{H}_{\text{goodcom}}$*).

Finish: When proceeding to the next round, for each role $R = (i, j) \in [m] \times [m]$ which is not yet assigned, the simulator inputs $(\text{NomCorrupt}, \text{cid}, \perp, \perp, \perp)$ to $\mathcal{F}$. It then sends $(\text{Finish}, \text{cid})$ to $\mathcal{F}$ on behalf of each honest party. Receive $(\text{Finish}, \text{cid}, \pi, \text{pkeys})$. In the subsequent round the simulator should reprogram the random beacon to match π .

D Lower Bounds

In this section, we provide lower bounds showing a connection between low communication and protocol resilience. One such bound by Rambaud [Ram23] has dealt with protocols in which parties are only allowed to multicast messages, i.e., send messages to all parties. Rambaud showed that every consensus protocol in the multicast model requires a linear number of multicasts if it has the optimal resilience of $n = 2t + 1$. We will say that a protocol is a *multicast protocol* if parties are only allowed to send multicasts during the protocol. Note that YOSO protocols are indeed multicast protocols, and thus, this lower bound captures a large class of current low-communication protocols. To strengthen the bound, we will consider consensus with only omission faults, in which up to t parties with omission failures can have both incoming and outgoing messages never be delivered. We remark that the bound provided below and that of [Ram23] show that in *any* protocol there must be a large enough number of parties that send

messages, not only in multicast protocols. However, when considering YOSO-style protocols, this means that there must be a large number of multicasts, as parties always multicast messages in such protocols.

Ramnaud’s lower bound works by essentially labeling some of the parties faulty without exhibiting any faults, rendering their inputs invalid, as no honest party started with their input. We extend this lower bound to additional natural notions of consensus, such as consensus with external validity [CKS05]. In consensus with external validity, parties have access to an oracle **valid** that allows parties to check whether values are considered valid outputs. Parties are then allowed to output any valid value, regardless of whether any honest party started with it as input. This notion has been used extensively in recent research, as often it is only important that parties agree on a value that can be used in the rest of the protocol (which can be checked by the validity oracle). Note that in regular consensus protocols, parties may output any value if they don’t start with unanimous agreement, which might not be useful in some settings. To compensate for weakening the validity requirement of the protocol, we strengthen its correctness requirements. We consider uniform consensus protocols, in which all parties that output values must agree, even if some of them are faulty.

We define consensus as follows:

Definition 2. *Let Π be a protocol for n parties $P_1, \dots, P_n$. Every party P_i has an input $x_i \in \{0, 1\}$ and may output a value $y_i \in \{0, 1\}$. Π is said to be a uniform consensus protocol with omission external validity if it has the following properties:*

- **External Validity.** *Honest parties only output some party’s input. Formally, if an honest party P_i outputs y_i , then there exists some $j \in [n]$ such that $x_j = y_i$.*
- **Uniform Agreement.** *All parties that output values from the protocol output the same value. Formally, if two parties P_i, P_j output y_i, y_j , then $y_i = y_j$.*
- **Termination.** *All honest parties eventually output values.*

Π is said to be $(1 - \epsilon)$ -correct if it has the above properties with probability $(1 - \epsilon)$ or greater, over the randomness of the adversary, the parties, and any setup.

We consider a simplified version of external validity in the omission case that simply says that every output has to be some party’s input. This captures the notion that to output y , parties “have to hear y from somebody”. Usually, in Byzantine agreement protocols with external validity, the external validity function checks that the value is valid cryptographically (e.g., is a correct signature on a message), and it is thus infeasible for parties to search for an externally valid value. As such, in practice, parties actually have to hear a value to output it, as is the case in the above definition.

In addition, uniform agreement protocols can be thought of as the non-Byzantine counterpart to *provable* consensus. In provable consensus, parties must also output certificates showing that the output is correct. This means that to output values, faulty parties must be able to produce information that shows

that they output the correct values. In other words, the information received by faulty parties must either allow them to provably output the correct value, or not allow them to output any value, like in uniform agreement.

D.1 Proof

Our goal is to show that for any uniform consensus protocol with external validity, there must be a relationship between the error probability, η , the number of parties in the network, $n = 2t + k$, and the number of parties that multicast messages B . We will partition the network into 3 groups, two large groups that do not communicate, and a small group that remains silent. This will allow us to execute a “split-brain” attack similar to that of [DLS84], by having two halves of the network not communicate with each other.

Concretely, we will prove that any protocol for $n = 2t + k$ parties with B expected multicasts must have an error probability $\epsilon \geq \frac{1}{2+1/(1-\xi)}$, with $\xi = B \cdot \frac{k}{t+k}$ and assuming $\xi < 1$. Note that as $\xi = B \cdot \frac{k}{t+k}$ approaches 0, the $\frac{1}{2+1/(1-\xi)}$ bound on ϵ approaches $\frac{1}{2}$. For example, setting B, k such that $B \cdot k = O(t)$, i.e., $k = O(\frac{t}{B})$, we get a constant error probability. In particular, setting $B = k = O(t^{\frac{1}{2}})$, we get $\xi = \frac{Bk}{t+k} = O(\frac{t}{t+k}) = O(1)$, and thus $\epsilon \geq \frac{1}{2+O(1)}$. We get a similar result setting $B = O(\text{polylog}(t))$, as is often the case in scalable protocols, unless k is at least $\Omega(\frac{t}{\text{polylog}(t)})$. That is, for a resilience of $n = (2+c)t = 2t + ct$, c must be very close to constant, as $k = ct$ must be at least $\Omega(\frac{t}{\text{polylog}(t)})$.

We will start by describing the 3 worlds used in our proof. For any partition $L \sqcup M \sqcup R = [n]$ with $|L| = |R| = f$ and $|M| = k$ (where $\sqcup$ denotes a disjoint union), we define four possible worlds in which the protocol can be run:

World	Inputs			Corrupted Set
1	$L \leftarrow 0,$	$M \leftarrow 0,$	$R \leftarrow 0$	R
2	$L \leftarrow 0,$	$M \leftarrow 1,$	$R \leftarrow 1$	L
3	$L \leftarrow 1,$	$M \leftarrow 1,$	$R \leftarrow 1$	L

In all worlds, messages between the honest and faulty parties are dropped, and all other messages are delivered instantly. We will first show that there is a high probability that no party in M sends messages in world 1. Since in worlds 1 and 2, parties in L have the same inputs, and have a high probability of not hearing from parties in M and R , they must behave similarly in both worlds. On the other hand, in worlds 2 and 3, parties in M and R have the same input and do not hear from parties in L . Therefore, their behavior is identical in both worlds. Finally, in world 1 parties must output 0 and in world 3 parties must output 1, resulting in a contradiction in world 2.

We will start by proving that there is indeed a large probability that no party in M sends a message in world 1:

Lemma 2. *Let there be a $(1 - \epsilon)$ -correct multicast uniform consensus protocol with omission external validity for $n = 2t + k$ parties such that the expected*

number of parties that multicasts messages in the protocol is at most B for some parameters ϵ, t, k, B .

There exists a partition $L^* \sqcup M^* \sqcup R^* = [n]$ with $|L^*| = |R^*| = t, |M^*| = k$ such that the probability that there exists an index $j \in M$ such that the party P_j sends messages in either world 1 is at most :

$$\xi \doteq B \frac{k}{t+k}.$$

Proof. We will start by defining the following random variables:

- For every partition L, M, R of $[n]$, $X(L, M, R)$ is the random variable that equals 1 if any party in M sends a message throughout an execution of world 1 with the partition L, M, R .
- For every partition L, M, R of $[n]$, and index $j \in [n]$, $Y(j, L, M, R)$ is the random variable that equals 1 if party P_j sends a message throughout an execution of world 1 with the partition L, M, R .

Our goal is to show that there exists a partition L^*, M^*, R^* such that:

$$\Pr[X(L^*, M^*, R^*) = 1] \leq \xi$$

We will do this by looking at all possible partitions, following a similar strategy to that of [Ram23]. For brevity, denote $\mathbf{parts} = \{(L, M, R) \text{ s.t. } |L| = |R| = t, |M| = k, L \sqcup M \sqcup R = [n]\}$.

$$\sum_{(L, M, R) \in \mathbf{parts}} \mathbb{E}[X(L, M, R)] \leq \sum_{(L, M, R) \in \mathbf{parts}} \sum_{j \in M} \mathbb{E}[Y(j, L, M, R)]$$

Note that this is true because by definition, if $X(L, M, R) = 1$, then for some $j \in M$, P_j sent a message, and thus $Y(j, L, M, R) = 1$ as well. In the expression $\sum_{(L, M, R) \in \mathbf{parts}} \sum_{j \in M} \mathbb{E}[Y(j, L, M, R)]$, for any set $S \subseteq [n]$ such that $|S| = t + k$, all parties in L, M have the exact same view in each partition where $L \cup M = S$, and thus act identically in all of these partitions. Also, for any S, j such that $|S| = t + k$ and $j \in S$, there are exactly $\binom{t+k-1}{k-1}$ choices of $M \subseteq S$ such that $j \in M$ and $|M| = k$. That is because after choosing one index to be in M , one needs to choose $k - 1$ more indices from $S \setminus \{j\}$ to include. With slight abuse of notation, we will denote $Y(j, S)$ to be the random variable that equals 1 if P_j sends messages when all parties in S are honest and have input 0, and all messages between honest and faulty parties are dropped. In other words:

$$\begin{aligned} \sum_{(L, M, R) \in \mathbf{parts}} \sum_{j \in M} \mathbb{E}[Y(j, L, M, R)] &= \sum_{S \subseteq [n] \text{ s.t. } |S| = t+k} \sum_{j \in S} \binom{t+k-1}{k-1} \mathbb{E}[Y(j, S)] \\ &\leq \binom{n}{t} \binom{t+k-1}{k-1} B, \end{aligned}$$

where the final inequality holds because the expected number of messages sent by parties equals the sum of the expectations that each party sends a message.

Combining the above inequalities, we get:

$$\sum_{(L,M,R) \in \mathbf{parts}} \mathbb{E}[X(L, M, R)] \leq \binom{n}{t} \binom{t+k-1}{k-1} B$$

Note that on the left-hand side, there are $\binom{n}{t}$ choices for L , and $\binom{t+k}{k}$ choices for M , given L , with 1 choice remaining for R . Since there are $\binom{n}{t} \binom{t+k}{k}$ summands, at least one of them must be smaller than the average, and thus there exists some partition $(L^*, M^*, R^*) \in \mathbf{parts}$ such that:

$$\begin{aligned} \Pr[X(L^*, M^*, R^*) = 1] &= \mathbb{E}[X(L^*, M^*, R^*)] \leq \binom{n}{t} \binom{t+k-1}{k-1} B / \left(\binom{n}{t} \binom{t+k}{k} \right) \\ &= \frac{k}{t+k} B = \xi, \end{aligned}$$

as required.

Using the above lemma, we can now prove our desired lower bound:

Theorem 3. *Let there be some parameters $\epsilon \in [0, 1]$, $k, t, B \in \mathbb{N}$, assuming $\xi \doteq B \frac{k}{t+k} < 1$.*

Let Π be an $(1-\epsilon)$ -correct multicast uniform consensus protocol with external validity for $n = 2t + k$ parties such that the expected number of parties that multicast messages is at most B . Then:

$$\epsilon \geq \frac{1}{2 + 1/(1-\xi)}.$$

Proof. Let Π be a protocol as described in the theorem statement, and let L, M, R be the partition defined in Lemma 2. Define G to be the “good event” that no party in M talks in an execution of world 1 with that partition. As shown in Lemma 2, $\Pr[G] \geq 1 - \xi$.

For every part of the partition $S \in \{L, M, R\}$ and world $i \in [3]$, define the random variable $O_{S,i}$ to be $b \in \{0, 1\}$ if all parties in S output b in an execution of the protocol in world i with the above partition. If the output is not unanimous, define S to be $\perp$. From the external validity of the protocol, in world 1, all honest parties must output 0 with probability $1 - \epsilon$ or greater. Similarly, in world 3, all honest parties must output 1 with probability $1 - \epsilon$ or greater.

In particular, this means that $\Pr[O_{L,1} = 0] \geq 1 - \epsilon$ and $\Pr[O_{R,3} = 1] \geq 1 - \epsilon$. Note that all parties in M and R have the exact same view in worlds 2 and 3, as in both worlds they all have input 1 and hear no messages from parties in L . Therefore, $\Pr[O_{R,2} = 1] \geq 1 - \epsilon$ as well. On the other hand, parties in L might have differing views in worlds 1 and 2. They have the same inputs in both worlds, but they might receive messages from parties in M in world 1, whereas they will never receive messages from them in world 2. We use Lemma 2 to show that there is a high probability that parties in L output 0 when they hear nothing

from the rest of the network.

$$\begin{aligned}
\Pr[O_{L,1} = 0] &= \Pr[O_{L,1} = 0|G] \cdot \Pr[G] + \Pr[O_{L,1} = 0|\bar{G}] \Pr[\bar{G}] \\
\implies \Pr[O_{L,1} = 0|G] &= \frac{\Pr[O_{L,1} = 0] - \Pr[O_{L,1} = 0|\bar{G}] \cdot \Pr[\bar{G}]}{\Pr[G]} \\
&\geq \frac{1 - \epsilon - \xi}{1 - \xi} = 1 - \frac{\epsilon}{1 - \xi}
\end{aligned}$$

In world 2, parties in L have the input 0 and hear nothing from the rest of the network, and thus must exhibit the same behavior as their behavior in world 1, given that no parties in M send messages. In other words: $\Pr[O_{L,2} = 0] = \Pr[O_{L,1} = 0|G] \geq 1 - \frac{\epsilon}{1 - \xi}$. Finally, observing world 2:

$$\begin{aligned}
\Pr[O_{L,2} = 0 \wedge O_{R,2} = 1] &= \Pr[O_{L,2} = 0] + \Pr[O_{R,2} = 1] - \Pr[O_{L,2} = 0 \vee O_{R,2} = 1] \\
&\geq (1 - \frac{\epsilon}{1 - \xi}) + (1 - \epsilon) - 1 = 1 - \epsilon - \frac{\epsilon}{1 - \xi}
\end{aligned}$$

Since in this event, different parties output different values in the same execution, the probability of the event must be bounded by ϵ . Therefore:

$$\begin{aligned}
\epsilon &\geq 1 - \epsilon - \frac{\epsilon}{1 - \xi} \\
\implies 2\epsilon + \frac{\epsilon}{1 - \xi} &\geq 1 \\
\implies \epsilon &\geq \frac{1}{2 + 1/(1 - \xi)},
\end{aligned}$$

as required.